Realización de consultas.

Caso práctico



Ana y **Juan** trabajan con una base de datos que contiene información sobre los **empleados de una empresa**, sus departamentos y distintos datos relacionados con su actividad profesional.

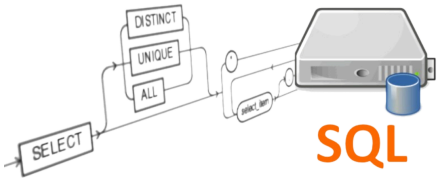
—Tenemos mucha información almacenada, pero no siempre necesitamos consultarla toda —comenta **Juan**.

—Exacto —responde **Ana**—. Con las **consultas SQL** podemos buscar únicamente los datos que nos interesan en cada momento y obtener resultados mucho más útiles.

A partir de estas consultas podrán **localizar información, relacionar datos y analizar resultados** de forma rápida, sin modificar la información almacenada en la base de datos.

Las bases de datos permiten **almacenar, organizar y recuperar información** de manera eficiente. Cuando la cantidad de datos aumenta, resulta necesario disponer de mecanismos que permitan localizar únicamente aquello que interesa y presentar los resultados con el nivel de **granularidad** adecuado.

Mediante SQL es posible realizar la **interrogación** de una base de datos, estableciendo diferentes criterios para obtener información concreta sin alterar los datos almacenados. Esto facilita el análisis de la información y permite aprovechar mejor los datos disponibles en distintos contextos profesionales.



campusmvp.es. *Relación de consultas.* (CC BY-NC-SA)

Preguntas para comenzar la unidad

1. ¿Qué **información sobre los empleados** crees que puede necesitar consultar habitualmente una empresa?
2. ¿Qué ventajas puede tener almacenar la información en **varias tablas relacionadas**?
3. ¿Por qué puede ser útil obtener únicamente los **datos necesarios** en cada consulta?
4. ¿Cómo puede ayudar una base de datos a **analizar información** y tomar mejores decisiones?

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN, FORMACIÓN PROFESIONAL Y DEPORTES

Materiales formativos de FP Online propiedad del Ministerio de Educación, Formación Profesional y Deportes.

Aviso Legal

1.- Introducción.

Caso práctico

Juan quiere comenzar con consultas básicas sobre los datos, obteniendo información concreta y relevante de cada una de las tablas. También necesita realizar algunos cálculos, como conocer el **salario medio de los empleados**, obtener el **mayor salario de cada departamento** o saber cuánto tiempo lleva cada empleado en la empresa.

Para conseguirlo tendrá que utilizar distintas **sentencias SQL**, comenzando por las consultas más sencillas antes de realizar operaciones más complejas con los datos.

SQL (Structured Query Language) es el lenguaje estándar utilizado para **definir, consultar y manipular datos** en bases de datos relacionales. Permite comunicarse con el **SGBD** mediante instrucciones para crear estructuras, recuperar información y modificar los datos almacenados.

SQL es un lenguaje utilizado para trabajar con bases de datos relacionales. Permite **crear estructuras, consultar información y modificar los datos almacenados**, por lo que constituye una de las principales vías de comunicación con un **sistema gestor de bases de datos**.

Su origen se encuentra en el **modelo relacional** propuesto por **Edgar F. Codd** en 1970. A partir de este modelo, IBM desarrolló inicialmente **SEQUEL (Structured English Query Language)**, que posteriormente pasó a denominarse **SQL (Structured Query Language)**.

SQL evolucionó hasta convertirse en un **estándar internacional**. Una de sus revisiones más conocidas fue **SQL-92** y posteriormente se han publicado nuevas versiones hasta llegar a **SQL:2023**, integrado en la norma **ISO/IEC 9075**. Los distintos SGBD pueden implementar el estándar en diferente grado y añadir además **características propias**.

Principales tipos de sentencias SQL

Las sentencias SQL pueden clasificarse según la operación que realizan:

- **DDL (Data Definition Language)** o **Lenguaje de Definición de Datos**: permite crear y modificar las **estructuras** de la base de datos. Incluye sentencias como **CREATE**, **ALTER** y **DROP**.
- **DML (Data Manipulation Language)** o **Lenguaje de Manipulación de Datos**: permite trabajar con la información almacenada mediante sentencias como **INSERT**, **UPDATE** y **DELETE**.
- **SELECT**: permite **consultar y recuperar datos** almacenados en una o varias tablas. Según la clasificación utilizada, puede considerarse parte de DML o incluirse dentro de **DQL (Data Query Language)**.

La sentencia **SELECT** será especialmente importante para obtener información de las tablas y realizar consultas de distinta complejidad.



Elaboración propia. *Tipos de sentencias de SQL.* (CC BY-NC-SA)

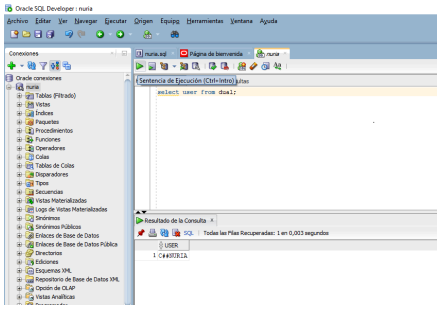
Ejecución de sentencias SQL

Las sentencias SQL pueden escribirse y ejecutarse mediante distintas herramientas.

Desde un entorno gráfico con SQL Developer

Oracle SQL Developer proporciona un entorno gráfico desde el que se pueden crear **conexiones**, explorar los objetos de una base de datos y escribir y ejecutar **sentencias y scripts SQL**.

Una vez establecida la conexión, las consultas se escriben en una **hoja de trabajo SQL** y sus resultados pueden visualizarse directamente en forma de tabla.



SQLDeveloper. Editor de SQL (CC BY-NC-SA)

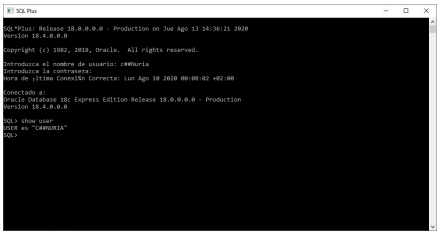
Desde una interfaz de línea de comandos

También es posible trabajar mediante **SQL*Plus**, la herramienta de línea de comandos de Oracle para ejecutar sentencias SQL y scripts.

Para utilizarla es necesario conectarse previamente mediante un **usuario** y su correspondiente **contraseña**. Ese usuario debe disponer de los **permisos necesarios** para realizar las operaciones solicitadas.

Las sentencias SQL deben respetar su **sintaxis** y normalmente finalizan con un **punto y coma (;)** antes de ejecutarlas.

En instalaciones de Oracle con arquitectura **multitenant**, determinados usuarios comunes pueden utilizar el prefijo **C##**, aunque este prefijo no es necesario para todos los tipos de usuario.



Oracle Corporation. *Conexión con SQLPlus* (CC BY-NC-SA)

Debes conocer

La descripción y el contenido de las tablas utilizadas en los ejemplos, junto con el **Sistema de Información**, el **DER** y su transformación al **modelo relacional**, se encuentran en el **Anexo I.- Base de datos de ejemplo (Juegos online).**

También se incluyen las sentencias necesarias para **crear las tablas** e **insertar datos de ejemplo**, lo que permitirá comprobar el funcionamiento de las consultas.

Conviene identificar correctamente las **claves primarias** y **claves ajenas** de cada tabla, ya que serán fundamentales al trabajar con información relacionada.

En el **Anexo II.- Creación y carga de tablas de la aplicación empresa en Oracle** se encuentran el script y los pasos necesarios para preparar las tablas de la aplicación **empresa**, utilizadas en los ejemplos y ejercicios prácticos.

Actividad desplegable

Una empresa comienza a trabajar con una base de datos Oracle. Relaciona cada necesidad con el concepto o herramienta más adecuada.

Situación	Respuesta correcta
1. crear o modificar la estructura de una tabla	
2. consultar los datos almacenados en una tabla	
3. escribir y ejecutar consultas desde un entorno gráfico	

Comprobar Mostrar retroalimentación

1. **ddl.** Correcto. **DDL** incluye las sentencias utilizadas para definir o modificar estructuras de la base de datos, como **CREATE** o **ALTER**.
2. **select.** Correcto. **SELECT** permite consultar y recuperar información almacenada en las tablas.
3. **sql developer.** Correcto. **SQL Developer** proporciona un entorno gráfico desde el que se pueden escribir y ejecutar sentencias SQL.

2.- La sentencia SELECT.

Caso práctico

Ana está trabajando con la tabla **PARTIDAS** y quiere identificar qué información resulta más útil para crear las consultas más sencillas y frecuentes. Sabe que mediante **SQL** y la sentencia **SELECT** puede aprovechar los datos almacenados en la tabla y obtener únicamente aquellos que necesite en cada momento.



La **sentencia SELECT** permite recuperar información de una o varias tablas, indicando **qué datos se desean obtener**, **de dónde proceden**, qué **condiciones** deben cumplir y cómo deben **ordenarse**.

¿Cómo podemos seleccionar los datos que nos interesan de una base de datos? Para **recuperar información** mediante SQL utilizamos la sentencia **SELECT**, cuya estructura puede incluir cuatro cláusulas principales:

- **SELECT**: indica las **columnas o expresiones** que queremos mostrar. Es la parte fundamental de la consulta.
- **FROM**: indica la **tabla o tablas** de las que proceden los datos. Se utiliza cuando la consulta obtiene información de tablas.
- **WHERE**: establece una **condición** para seleccionar únicamente las filas que la cumplen. Es opcional.
- **ORDER BY**: establece el **criterio de ordenación** de los resultados. Es opcional.

Por tanto, una primera sintaxis quedaría de la siguiente forma:

```
SELECT [ALL | DISTINCT] columna1, columna2, ...
FROM tabla1, tabla2, ...
WHERE condición
ORDER BY columna [ASC | DESC];
```

*Las palabras **ALL** y **DISTINCT** son opcionales:*

- **ALL** indica que se mostrarán todas las filas obtenidas, aunque existan valores repetidos. Es el comportamiento predeterminado y normalmente no es necesario escribirlo.
- **DISTINCT** elimina del resultado las **filas duplicadas** teniendo en cuenta las columnas seleccionadas.

Por ejemplo, si una consulta devuelve varias filas con exactamente los mismos valores en las columnas seleccionadas, **DISTINCT** hará que esa combinación aparezca una sola vez.

Recuerda

En la representación de la **sintaxis SQL** pueden aparecer símbolos que ayudan a explicar cómo debe construirse una sentencia, pero que no siempre se escriben literalmente:

- Los **corchetes [ ]** indican que un elemento es **opcional**.
- La **barra vertical |** indica que debe elegirse entre distintas alternativas.
- Los **puntos suspensivos ...** indican que pueden añadirse más elementos del mismo tipo.

Ejemplo:

```
SELECT [ALL | DISTINCT] columna1, columna2, ... FROM tabla1, tabla2, ... WHERE condición ORDER BY columna;
```

Esto indica que **ALL** o **DISTINCT** son opcionales y alternativos, y que pueden especificarse varias columnas, tablas o condiciones según las necesidades de la consulta.

Autoevaluación

Una aplicación necesita obtener de la tabla **USUARIOS** únicamente los valores diferentes de la columna provincia y mostrarlos ordenados alfabéticamente.

¿Qué consulta responde correctamente a este requisito?

- ○ a) select all provincia from usuarios order by provincia;
- ○ b) select provincia from usuarios where distinct provincia order by provincia;
- ○ c) select distinct provincia from usuarios order by provincia;

Incorrecta. all mantiene todas las filas obtenidas, incluidas las repeticiones, por lo que una provincia podría aparecer varias veces.No, éstas irían junto a la cláusula SELECT.

Incorrecta. distinct no se utiliza dentro de la cláusula **where**. Debe escribirse después de **select**.

Correcta. distinct elimina los valores repetidos de provincia y **order by** ordena el resultado por esa misma columna.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

2.1.- Cláusula SELECT.

La **cláusula SELECT** determina qué **columnas, expresiones, constantes o funciones** aparecerán en el resultado de una consulta y permite utilizar **alias** para presentar la información de forma más clara.

Ya has visto que a continuación de la sentencia **SELECT** debemos especificar cada una de las columnas que queremos seleccionar. Además, debemos tener en cuenta lo siguiente:

- **Se pueden nombrar** (o calificar) a las columnas anteponiendo el nombre de la tabla de la que proceden, pero esto es opcional solo si no existe el mismo nombre de columna en dos tablas Se realiza anteponiendo el nombre de la tabla a la que pertenece la columna seguida por un punto, es decir, NombreTabla.NombreColumna.
- Si queremos **incluir todas las columnas** de una tabla podemos utilizar el comodín **asterisco** ("*"). Quedaría así: **SELECT * FROM NombreTabla;**
- También podemos **ponerle alias a los nombres** de las columnas. Cuando se consulta una base de datos, los nombres de las columnas se usan como cabeceras de presentación. Si éste resulta largo, corto o poco descriptivo, podemos usar un alias. Para ello a continuación del nombre de la columna ponemos entre comillas dobles el alias que demos a esa columna. Veamos un ejemplo:

```
SELECT F_Nacimiento "Fecha de Nacimiento" FROM USUARIOS;
```

También podemos **sustituir el nombre** de las columnas por constantes, expresiones o funciones SQL. Un ejemplo:

```
SELECT 4*3/100 "MiExpresión", Password FROM USUARIOS;
```

Para saber más

Si quieres conocer algo más sobre esta sentencia y ver algunos ejemplos del uso de SELECT aquí tienes el siguiente enlace:

La cláusula SELECT.

Autoevaluación

Completa la consulta seleccionando la opción adecuada para cada hueco:

SELECT _____, salario * 12 _____ FROM EMPLEADOS;

Opciones:

a. Nombre
b. As salario_anual
c. Empleados

El objetivo es mostrar el nombre del empleado y su salario anual con la cabecera **salario_anual**.

SELECT [ ] , salario * 12 [ ] FROM EMPLEADOS;

Enviar

a. **Nombre.** Correcto para el primer hueco. Indica una de las **columnas** que se desea mostrar en el resultado.
b. **As salario_anual.** Correcto para el segundo hueco. Permite asignar un **alias** al resultado de la expresión salario * 12.
c. **Empleados.** No corresponde a ninguno de esos huecos. **EMPLEADOS** es el nombre de la tabla y debe aparecer después de la cláusula **FROM**.

2.2.- Cláusula FROM.

La **cláusula FROM** indica la **tabla o tablas de origen** de las que se obtienen los datos utilizados en una consulta.

Al realizar una consulta podemos elegir las columnas que necesitamos, pero también debemos indicar **de dónde procede la información**. Para ello se utiliza la cláusula **FROM**.

En una sentencia **SELECT**, después de **FROM** se especifican los nombres de las **tablas** de las que proceden los datos.

Si intervienen varias tablas, pueden combinarse mediante **JOIN**, indicando la condición que permite relacionarlas.

*También puede indicarse el **esquema o usuario propietario** de una tabla mediante la forma:*

USUARIO.TABLA

De este modo pueden distinguirse tablas con el mismo nombre pertenecientes a **esquemas diferentes**.

Además, es posible asignar un **alias** a una tabla para abreviar su nombre dentro de la consulta. En su uso habitual, el alias no necesita escribirse entre comillas.

Ejemplo:

```
SELECT * FROM USUARIOS U;
```

```
SELECT LOGIN, NOMBRE, APELLIDOS FROM USUARIOS;
```

En ambos casos se consultan todas las filas de **USUARIOS**. El primero muestra **todas las columnas** y el segundo únicamente **LOGIN, NOMBRE y APELLIDOS**.



Actividad desplegable

Una aplicación utiliza distintas formas de indicar el origen de los datos en sus consultas. Relaciona cada expresión con su significado.

expresión	significado
	1. utiliza directamente una tabla como origen de los datos
	2. identifica la tabla indicando también el esquema al que pertenece
	3. asigna un alias o nombre abreviado a la tabla

Comprobar | Mostrar retroalimentación

1. from usuarios → utiliza directamente una tabla como origen de los datos. Correcto. La cláusula **FROM** indica que los datos se obtienen de la tabla **USUARIOS**.

2. from empresa.usuarios → identifica la tabla indicando también el esquema al que pertenece. Correcto. La forma **esquema.tabla** permite indicar de forma precisa dónde se encuentra la tabla.

3. from usuarios u → asigna un alias o nombre abreviado a la tabla. Correcto. **u** funciona como **alias** de USUARIOS y permite referirse a la tabla de forma más breve dentro de la consulta.

2.3.- Cláusula WHERE.

La **cláusula WHERE** permite **filtrar las filas** de una consulta mediante una o varias **condiciones**, mostrando únicamente los registros que cumplen los criterios establecidos.

Hasta ahora hemos utilizado **SELECT** para obtener todas las filas de una tabla. Sin embargo, normalmente necesitaremos recuperar solo aquellos registros que cumplan una **condición determinada**.

Para ello se utiliza la cláusula **WHERE**. A continuación de esta palabra se especifica el **criterio de búsqueda** que deben cumplir las filas para formar parte del resultado.

Las condiciones pueden construirse utilizando **operadores de comparación**, **operadores lógicos**, funciones o expresiones.

Ejemplo:

Si queremos obtener el nombre y los apellidos de las usuarias cuyo valor de sexo es M :

```
SELECT nombre, apellidos
FROM USUARIOS
WHERE sexo = 'M';
```

En este caso, **WHERE** evalúa cada fila y únicamente devuelve aquellas en las que el campo **sexo** contiene el valor indicado.



Autoevaluación

Una aplicación realiza consultas sobre la tabla **USUARIOS**. Indica si las siguientes afirmaciones son verdaderas o falsas.

1. la cláusula where permite decidir qué filas formarán parte del resultado según una condición.

○ Verdadero ○ Falso

Verdadero

WHERE permite filtrar los registros mediante una condición, por lo que solo se muestran las filas que cumplen el criterio establecido.

2. la cláusula where se utiliza para escoger las columnas que aparecerán en el resultado.

○ Verdadero ○ Falso

Falso

la selección de columnas se realiza mediante **SELECT**. **WHERE** se utiliza para filtrar las filas según una condición.

3. una condición de where puede construirse utilizando operadores, funciones o expresiones.

○ Verdadero ○ Falso

Verdadero

Las condiciones pueden construirse combinando distintos **operadores, funciones y expresiones** para realizar filtros más precisos.

2.4.- Ordenación de registros. Cláusula ORDER BY.

La cláusula **ORDER BY** permite **ordenar las filas obtenidas en una consulta** según una o varias columnas o expresiones, de forma **ascendente (ASC)** o **descendente (DESC)**.

En la consulta anterior se obtenían los nombres y apellidos de determinados usuarios. Para facilitar la **lectura y localización de los resultados**, puede ser útil mostrarlos siguiendo un orden concreto. Para ello se utiliza la cláusula **ORDER BY**.



Stockbyte. (Uso educativo nc)

Su sintaxis básica es:

```
SELECT [ALL | DISTINCT] columna1, columna2, ...
FROM tabla
[WHERE condición]
ORDER BY columna1 [ASC | DESC], columna2 [ASC | DESC], ...;
```

Después de cada criterio de ordenación puede indicarse:

- **ASC**: orden ascendente. Es la opción **predeterminada**, por lo que puede omitirse.
- **DESC**: orden descendente.

Es posible **ordenar por más de una columna**. En ese caso, SQL utiliza primero la primera columna y, cuando existen valores iguales, aplica la siguiente.

Ejemplo:

Para ordenar los usuarios primero por **apellidos** y después por **nombre**:

```
SELECT nombre, apellidos
FROM USUARIOS
ORDER BY apellidos, nombre;
```

También se puede ordenar utilizando la **posición de una columna en la lista SELECT**:

```
SELECT nombre, apellidos, localidad
FROM usuarios
ORDER BY 3;
```

En este ejemplo, **3** hace referencia a **localidad**, que ocupa la tercera posición de la lista de selección. Si se utiliza una posición que no existe, la consulta producirá un **error**.

Aunque esta posibilidad es válida, utilizar el **nombre de la columna o la expresión** suele hacer que la consulta resulte más clara y fácil de mantener.

Además de columnas, **ORDER BY** puede utilizar **expresiones** o elementos calculados que puedan ordenarse correctamente.



Ejercicio resuelto

Utilizando las tablas de la aplicación **EMPRESA**, se quiere obtener de la tabla **ESTUDIOS** el **DNI de los empleados**, ordenando los resultados primero por **universidad de forma descendente** y, cuando coincida la universidad, por **año de forma ascendente**.

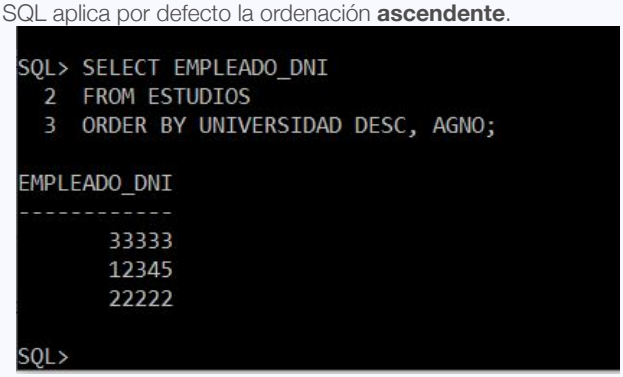
La columna correspondiente al año se denomina **AGNO**.

¿Qué consulta SQL permite obtener este resultado aplicando correctamente ambos criterios de ordenación?

Mostrar retroalimentación

```sql
SELECT EMPLEADO_DNI
FROM ESTUDIOS
ORDER BY UNIVERSIDAD DESC, AGNO;
```

La columna **UNIVERSIDAD** se ordena de forma descendente mediante **DESC**. Para **AGNO** no se especifica ningún criterio, por lo que SQL aplica por defecto la ordenación **ascendente**.



3.- Operadores.

Caso práctico

Ana y **Juan** necesitan realizar consultas que cumplan **criterios concretos**, como obtener usuarios con una determinada cantidad de créditos o combinar varias condiciones.

—Podemos filtrar los datos con **WHERE**, pero necesitamos algo más para construir bien esas condiciones —comenta **Juan**.

—Para eso utilizaremos distintos **operadores** —responde **Ana**—. Con ellos podremos comparar valores, hacer cálculos, unir textos y combinar condiciones.

Así queda más natural, mantiene la historia continua y sigue siendo **breve y visual**.



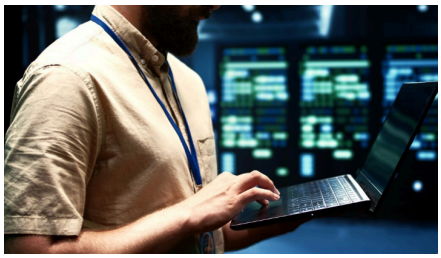
Stockbyte. (Uso educativo nc)

Los **operadores SQL** son **símbolos** o **palabras reservadas** que permiten realizar **comparaciones, cálculos, concatenaciones y operaciones lógicas** dentro de expresiones y condiciones.

En la cláusula **WHERE** pueden utilizarse expresiones para **filtrar los datos** que queremos obtener. Para construirlas se emplean diferentes operadores que permiten comparar valores, realizar cálculos, unir cadenas o combinar varias condiciones.

Los principales tipos de operadores son:

1. **Relacionales o de comparación:** permiten **comparar valores**, por ejemplo para comprobar si son iguales, distintos, mayores o menores.
2. **Aritméticos:** permiten realizar **operaciones matemáticas** como sumas, restas, multiplicaciones o divisiones.
3. **De concatenación:** permiten **unir cadenas de caracteres** para obtener un único resultado.
4. **Lógicos:** permiten **combinar o modificar condiciones**, utilizando operadores como **AND, OR** y **NOT**.



DC Studio. *Operadores.* (CC BY-NC-SA)

La elección del operador dependerá del **tipo de dato** y del resultado que se quiera obtener en la consulta.

Reflexiona

Una aplicación necesita obtener los usuarios que tengan **más de 200 créditos** y que, además, sean de una determinada provincia. Otro desarrollador propone realizar primero una consulta para comprobar los créditos y después una segunda consulta para comprobar la provincia.

1. ¿Sería necesario realizar dos consultas diferentes?

2. ¿Qué tipos de operadores permitirían expresar ambos requisitos dentro de una misma condición?

Mostrar retroalimentación

No sería necesario realizar dos consultas. En una misma condición pueden combinarse distintos **operadores**.

Para comprobar que el crédito supera un valor concreto se utilizaría un **operador de comparación**, mientras que para exigir simultáneamente que también se cumpla la condición de la provincia se utilizaría un **operador lógico**, como **AND**.

De esta forma, una única consulta puede evaluar **varios criterios a la vez** y devolver únicamente las filas que cumplen todos ellos.

3.1.- Operadores de comparación.

Los **operadores de comparación** permiten comparar valores o expresiones y obtener un resultado **verdadero o falso**, siendo fundamentales para establecer condiciones en cláusulas como **WHERE**.



DC Studio. *Operadores de comparación.* (CC BY-NC-SA)

Los operadores de comparación, también llamados **operadores relacionales**, permiten comparar valores de columnas, constantes o expresiones. Si la comparación se cumple, el resultado es **verdadero**; en caso contrario, es **falso**.

Los principales son:

Operador	Significado
=	igual a
!=, <>, ^=	distinto de
<	menor que
>	mayor que
<=	menor o igual que
>=	mayor o igual que
IN	comprueba si un valor coincide con alguno de los valores indicados
NOT IN	comprueba si un valor no coincide con ninguno de los valores indicados
BETWEEN	comprueba si un valor se encuentra dentro de un **rango**, incluyendo sus límites
NOT BETWEEN	comprueba si un valor se encuentra fuera de un rango
LIKE	permite buscar valores que cumplen un **patrón de texto**
IS NULL	comprueba si un valor es **NULL**
IS NOT NULL	comprueba si un valor no es **NULL**

*Con **LIKE** pueden utilizarse caracteres comodín:*

- **%** representa cero, uno o varios caracteres.
- _ representa exactamente un carácter.

El valor **NULL** representa un valor desconocido o inexistente. Por ello, no debe compararse mediante **=** o **<>**, sino utilizando **IS NULL** o **IS NOT NULL**.

> **Importante:** El orden de los valores **NULL** en una consulta con **ORDER BY** puede depender del SGBD. En Oracle puede controlarse explícitamente mediante **NULLS FIRST** o **NULLS LAST**, evitando depender del comportamiento predeterminado.

Ejemplo:

Para obtener los empleados cuyo **salario sea superior a 1000**:

Si queremos obtener aquellos empleados cuyo salario es superior a 1000€ podemos crear la siguiente consulta:

```
SELECT nombre FROM EMPLEADOS WHERE SALARIO > 1000;
```

Para obtener los empleados cuyo **primer apellido comienza por R**:

```
SELECT nombre FROM EMPLEADOS WHERE APELLIDO1 LIKE 'R %';
```

Para saber más

En el siguiente enlace tienes un completo manual de SQL de Oracle explicado de forma sencilla y preparado para hacer ejercicios online. Te será útil para practicar.

Tutorial SQL Oracle

Las **expresiones regulares** permiten forman un patrón, normalmente representativo de otro grupo de carácteres mayor, para comparar el patrón con otro conjunto de carácteres para ver las coincidencias.Es una herramienta potente para comparar cadenas que cumplan un determinado patrón, por ejemplo para determinar si un email está bien compuesto.

En el siguiente enlace tienes toda la información con ejemplos:

Uso de expresiones regulares

Los operadores que se utilizan en MySQL puedes verlos en el siguiente enlace:

Operadores de comparación en MySQL.

Ejercicio resuelto

Utilizando las tablas y datos de la aplicación **EMPRESA**, se necesita obtener el **código y el nombre de las universidades** situadas en **Sevilla o Madrid**.

¿Qué consulta permite seleccionar ambas ciudades utilizando un único operador de comparación?

Mostrar retroalimentación

```
SELECT UNIV_COD, NOMBRE_UNIV
FROM UNIVERSIDADES
WHERE CIUDAD IN ('SEVILLA', 'MADRID');
```

El operador **IN** permite comprobar si el valor de **CIUDAD** coincide con cualquiera de los valores incluidos en la lista. De esta forma, la consulta selecciona las universidades cuya ciudad sea **Sevilla o Madrid**.

```
SQL> SELECT UNIV_COD, NOMBRE_UNIV FROM UNIVERSIDADES WHERE CIUDAD IN ('SEVILLA', 'MADRID');

  UNIV_COD NOMBRE_UNIV
---------- ------------------------
         1 UNED
         2 SEVILLA

SQL>
```

3.2.- Operadores aritméticos y de concatenación.

Los **operadores aritméticos** permiten realizar **cálculos** con **valores numéricos,** mientras que el operador de **concatenación** permite **unir** varios **textos o valores** en una única cadena.

Los **operadores aritméticos** permiten realizar operaciones matemáticas con valores numéricos:

operador	significado
+	suma
-	resta
*	multiplicación
/	división

Mediante estos operadores es posible crear **expresiones calculadas**, de forma que una columna del resultado proceda de un cálculo y no directamente de un campo almacenado en la tabla.

Ejemplo:

Para mostrar el salario aumentado un **5 %** de los trabajadores que cobran 1000 € o menos:

```
SELECT SALARIO*1,05
FROM TRABAJADORES
WHERE SALARIO<=1000;
```

Cuando una **expresión aritmética** opera con un valor **NULL**, el resultado de la expresión será también **NULL**.

Operador de concatenación

Para unir **cadenas de caracteres**, Oracle utiliza el operador de concatenación **||**.

Ejemplo:

Si en la tabla EMPLEADOS el primer y segundo apellido están almacenados en columnas diferentes, pueden mostrarse juntos:

```
SELECT Nombre, Apellido1 || Apellido2
FROM EMPLEADOS;
```

Para mejorar la presentación, podemos concatenar también un **espacio en blanco** y asignar un **alias** al resultado:

```
SELECT Nombre, Apellido1 || ' ' ||Apellido2 Apellidos
FROM EMPLEADOS;
```



Para saber más

Los operadores que se utilizan en MySQL puedes verlos en el siguiente enlace:

Operadores aritméticos en MySQL.



Autoevaluación

Una empresa quiere mostrar en una consulta el **nombre y los dos apellidos de cada empleado en una sola columna**, separados por espacios.

¿Qué expresión realiza correctamente esa operación en Oracle?

- ○ a. nombre + ' ' + apellido1 + ' ' + apellido2
- ○ b. nombre || ' ' || apellido1 || ' ' || apellido2
- ○ c. concat(nombre, apellido1, apellido2)

Incorrecta. En Oracle, el operador **+** se utiliza para operaciones aritméticas, no para concatenar cadenas.

Correcta. El operador **||** permite unir varias cadenas y columnas de texto, incluyendo espacios entre ellas.

Incorrecta. En Oracle, **CONCAT** admite dos argumentos cada vez, por lo que esa expresión con tres argumentos no sería válida.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

3.3.- Operadores lógicos.

Los **operadores lógicos** permiten combinar o negar varias **condiciones** dentro de una consulta para decidir qué registros cumplen los criterios establecidos.

Habrá ocasiones en las que sea necesario evaluar **más de una condición** dentro de una consulta. Para combinarlas se utilizan los operadores lógicos **AND**, **OR** y **NOT**.

operador	significado
AND	devuelve verdadero cuando **todas las condiciones** combinadas se cumplen
OR	devuelve verdadero cuando se cumple **al menos una de las condiciones**
NOT	**niega una condición**, invirtiendo su resultado lógico

Cuando intervienen valores **NULL**, SQL puede obtener también un resultado lógico **desconocido (UNKNOWN)**, por lo que deben tratarse adecuadamente mediante operadores como **IS NULL** o **IS NOT NULL**.

Ejemplo:

Si queremos obtener los empleados cuyo salario sea **menor o igual a 800 € o superior a 2000 €**:

```
SELECT empleado_dni
FROM HISTORIAL_SALARIAL
WHERE salario <=800 OR salario>2000;
```

El operador **OR** hace que el registro sea seleccionado cuando se cumple **cualquiera de las dos condiciones**.



Para saber más

Si quieres conocer un poco más sobre los operadores lógicos visita este enlace:

Operadores lógicos.



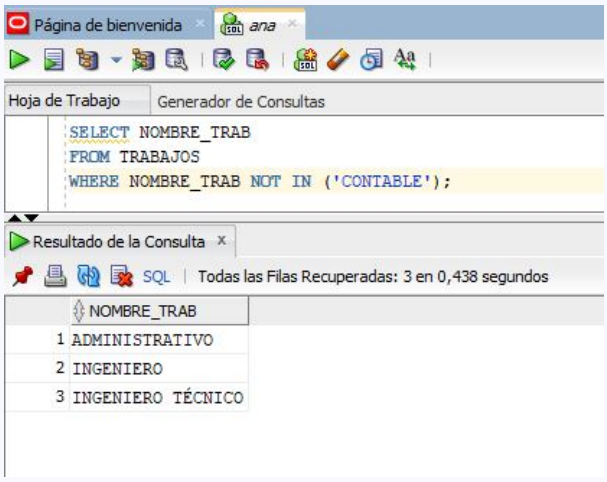
Ejercicio resuelto

Utilizando las tablas y datos de la aplicación **EMPRESA**, se necesita obtener todos los **nombres de los trabajos excepto CONTABLE**.

¿Qué condición permite excluir del resultado el trabajo denominado CONTABLE?

Mostrar retroalimentación

Escribimos la sentencia en SQLDeveloper



Ministerio de Educación - Uso educativo nc

La expresión **NOT IN** permite excluir los valores incluidos en la lista. En este caso, se muestran todos los trabajos cuyo nombre **no sea CONTABLE**.

3.4.- Precedencia.

La **precedencia de operadores** determina el **orden en que SQL evalúa las operaciones y condiciones** de una expresión cuando no se utilizan paréntesis.

Cuando una sentencia **SELECT** contiene expresiones complejas, es importante saber qué operación se evaluará primero. SQL aplica un **orden de precedencia** y, cuando varios operadores tienen el mismo nivel, normalmente se evalúan según el orden establecido para ese tipo de expresión.



Stockbyte.. *Precedencia.* (CC BY-NC-SA)

De forma general, el orden es:

1. **Multiplicación (*) y división (/)**.
2. **Suma (+) y resta (-)**.
3. **Concatenación (||)**.
4. **Comparaciones**: =, !=, <>, <, >, <=, >=.
5. **IS NULL, IS NOT NULL, LIKE, IN y BETWEEN**.
6. **NOT**.
7. **AND**.
8. **OR**.

Si queremos **modificar este orden**, podemos utilizar **paréntesis**, que obligan a evaluar primero la expresión que contienen.

Ejemplo:

```
WHERE salario > 1500 AND departamento = 'VENTAS' OR antiguedad > 10
```

En esta condición se evaluará **AND antes que OR**. Si queremos que primero se compruebe una condición con OR, debemos indicarlo mediante paréntesis.

Autoevaluación

En la siguiente consulta:

```
SELECT APELLIDOS
FROM JUGADORES
WHERE APELLIDOS LIKE 'A%S%';
```

¿Qué estaríamos seleccionando?

- ○ a) Aquellos jugadores cuyos apellidos contienen la letra A y la S.
- ○ b) Aquellos jugadores cuyos apellidos comienzan por la letra A y contienen la letra S.
- ○ c) Aquellos jugadores cuyos apellidos no contienen ni la letra A ni la S.
- ○ d) Todos los apellidos de todos los jugadores menos los que su apellido comienza por S.

Incorrecta. El patrón **'A%S%'** no busca simplemente apellidos que contengan las letras A y S en cualquier posición. La **A debe aparecer al comienzo** del apellido.

Correcta. El patrón **'A%S%'** indica que el apellido debe **comenzar por A**, después puede contener cualquier conjunto de caracteres mediante **%**, debe aparecer una **S** posteriormente y puede haber más caracteres después.

Incorrecta. La consulta busca precisamente apellidos que cumplan el patrón indicado; no excluye las letras A y S.

Incorrecta. La consulta no selecciona todos los apellidos excepto los que comienzan por S. Solo devuelve aquellos que **empiezan por A y contienen una S después**.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

4.- Consultas calculadas.

Caso práctico

A la empresa ha llegado **Carlos**, que está realizando sus prácticas y comienza a colaborar con **Juan**. Necesita recordar cómo obtener valores nuevos a partir de los datos que ya existen en las tablas.



—Si necesito mostrar un precio con **IVA,** ¿tengo que guardar ese nuevo valor en la tabla? —pregunta **Carlos**.

—No necesariamente —responde **Juan**—. Podemos calcularlo directamente en la consulta sin modificar los datos originales.

Para practicar, **Carlos** utilizará campos numéricos como **Precio** y **Cantidad**, con los que podrá calcular precios con IVA, descuentos o incrementos porcentuales.

Las **consultas calculadas** permiten obtener nuevos valores mediante **expresiones y operaciones realizadas sobre los datos existentes**, sin modificar la información almacenada en las tablas.

En ocasiones necesitamos realizar **operaciones sobre los datos** para obtener información derivada. Si tenemos un precio, podemos calcular su importe con IVA; si disponemos de un sueldo y una paga extra, podemos obtener la suma de ambos.



learnsql.es. *Consultas calculadas* (CC BY-NC-SA)

Los **operadores aritméticos** permiten construir estas expresiones directamente dentro de una sentencia **SELECT**.

Las **columnas calculadas** solo aparecen en el resultado de la consulta: **no modifican los valores originales** almacenados en la tabla.

Ejemplo:

Para mostrar el crédito actual de cada usuario y el resultado de aumentarlo en 25:

```
SELECT Nombre, Credito, Credito + 25
FROM USUARIOS;
```

Si no se especifica ningún nombre, la columna calculada puede aparecer identificada mediante la propia expresión. Para hacer el resultado más comprensible, se puede asignar un **alias** mediante **AS**:

```
SELECT Nombre, Credito, Credito + 25 AS CreditoNuevo
FROM USUARIOS;
```

De esta forma, **CreditoNuevo** será únicamente el nombre mostrado para la columna calculada y el valor original de **Credito permanecerá sin cambios**.

Reflexiona

Una tienda almacena en la tabla PRODUCTOS el **precio sin IVA** de cada artículo. Para mostrar el precio final en su catálogo, un desarrollador propone añadir una nueva columna llamada precio_con_iva y actualizarla cada vez que cambie el precio original.

1. ¿Es necesario almacenar ese nuevo valor?

2. ¿Qué ventaja tendría calcularlo directamente mediante una consulta?

Mostrar retroalimentación

No sería necesario almacenar **precio_con_iva** si puede obtenerse a partir del **precio original** mediante una expresión.

Una **consulta calculada** permitiría obtener el precio final cuando fuera necesario, sin modificar ni duplicar los datos almacenados. De esta forma se evita que ambos precios puedan quedar **inconsistentes** si cambia el valor original y no se actualiza el calculado.

5.- Funciones.

Caso práctico

Mientras **Carlos** continúa practicando con las consultas calculadas, **Juan** le pide a **Ana** que obtenga la edad de los usuarios registrados para poder realizar estadísticas según sus grupos de edad.

—Calcularlo manualmente para cada usuario no tendría sentido —comenta **Ana**.

—Seguro que SQL tiene alguna forma de hacerlo automáticamente —responde **Carlos**.

Para resolver este tipo de situaciones utilizarán **funciones SQL**, capaces de realizar cálculos y transformaciones sobre los datos.



Stockbyte (Uso educativo nc)

Las **funciones** son operaciones incorporadas por los SGBD que permiten realizar determinados **cálculos o transformaciones** sobre los datos. Las funciones disponibles y su comportamiento pueden variar según el **SGBD** utilizado.

Para realizar su operación suelen recibir uno o varios datos de entrada denominados **parámetros o argumentos** y devuelven un resultado.

Su formato habitual es:

```
NombreFunción(parámetro1, parámetro2, ...)
```

Las funciones pueden utilizarse dentro de distintas expresiones SQL, como las incluidas en **SELECT**, **WHERE** u **ORDER BY**. Además, es posible **anidar funciones**, utilizando el resultado de una como argumento de otra.

Según el tipo de operación que realizan, podemos encontrar funciones:

- **Numéricas.**
- De **cadenas de caracteres.**
- De **fechas y horas.**
- De **conversión.**
- Y **otras funciones** específicas del SGBD.

La tabla DUAL en Oracle

Oracle dispone de la tabla **DUAL**, formada por una única columna denominada **DUMMY** y una sola fila. Se utiliza habitualmente para probar **funciones, constantes o expresiones** cuando no es necesario consultar una tabla con datos.

Desde **Oracle Database 23**, la cláusula **FROM DUAL** puede omitirse cuando se seleccionan expresiones que no necesitan una tabla. Por tanto, pueden encontrarse tanto consultas con **FROM DUAL** como sin ella.

Ejemplo:

```
SELECT SYSDATE FROM DUAL;
```

También puede escribirse, en versiones actuales de Oracle:

```
SELECT SYSDATE;
```

Reflexiona

Una aplicación almacena la fecha de nacimiento de sus usuarios. Para mostrar su edad, un desarrollador propone añadir una nueva columna y actualizarla cada año, mientras que otro propone obtenerla mediante funciones SQL cuando se

realice la consulta.

¿Qué opción consideras más adecuada y qué ventaja tendría utilizar funciones para obtener la edad?

Mostrar retroalimentación

Resulta más adecuado conservar la **fecha de nacimiento** y calcular la edad cuando sea necesaria mediante las **funciones de fecha** correspondientes.

La edad es un valor que **cambia con el tiempo**, por lo que almacenarla obligaría a actualizarla periódicamente y podría provocar datos incorrectos. Utilizar funciones permite obtener un **valor calculado y actualizado** sin modificar ni duplicar la información almacenada.

5.1.- Funciones numéricas.

Las **funciones numéricas** permiten realizar cálculos y operaciones sobre valores numéricos, como obtener el **valor absoluto**, redondear cantidades o calcular el **resto de una división**.

¿Cómo obtenemos el cuadrado de un número o su valor absoluto? Nos referimos a valores numéricos y por tanto necesitaremos utilizar funciones numéricas.



StockByte - Uso educativo nc. *Funciones numéricas.* (CC BY-NC-SA)

Para trabajar con campos de tipo número tenemos las siguientes funciones:

- **ABS(n)**
  Calcula el valor absoluto de un número n.

  Ejemplo:

```
SELECT ABS(-17) FROM DUAL; -- Resultado: 17
```

- **EXP(n)**
  Calcula e^n , es decir, el exponente en base e del número n.

  Ejemplo:

```
SELECT EXP(2) FROM DUAL; -- Resultado: 7,38
```

- **CEIL(n)**
  Calcula el valor entero inmediatamente superior o igual al argumento n.

  Ejemplo:

```
SELECT CEIL(17.4) FROM DUAL; -- Resultado: 18
```

- **FLOOR(n)**
  Calcula el valor entero inmediatamente inferior o igual al parámetro n.

  Ejemplo:

```
SELECT FLOOR(17.4) FROM DUAL; -- Resultado: 17
```

- **MOD(m,n)**
  Calcula el resto resultante de dividir m entre n.

  Ejemplo:

```
SELECT MOD(15, 2) FROM DUAL; --Resultado: 1
```

- **POWER(valor, exponente)**
  Eleva el valor al exponente indicado.

  Ejemplo:

```
SELECT POWER(4, 5) FROM DUAL; -- Resultado: 1024
```

- **ROUND(n, decimales)**
  Redondea el número n al siguiente número con el número de decimales que se indican.

  Ejemplo:

```
SELECT ROUND(12.5874, 2) FROM DUAL; -- Resultado: 12.59
```

SQRT(n)
Calcula la raíz cuadrada de n.

Ejemplo:

```
SELECT SQRT(25) FROM DUAL; --Resultado: 5
```

TRUNC(m,n)
Trunca un número a la cantidad de decimales especificada por el segundo argumento. Si se omite el segundo argumento, se truncan todos los decimales. Si "n" es negativo, el número es truncado desde la parte entera.

Ejemplos:

```
SELECT TRUNC(127.4567, 2) FROM DUAL; -- Resultado: 127.45
SELECT TRUNC(4572.5678, -2) FROM DUAL; -- Resultado: 4500
SELECT TRUNC(4572.5678, -1) FROM DUAL; -- Resultado: 4570
SELECT TRUNC(4572.5678) FROM DUAL; -- Resultado: 4572
```

SIGN(n)
Si el argumento "n"es un valor positivo, retorna 1, si es negativo, devuelve -1 y 0 si es 0.

```
SELECT SIGN(-23) FROM DUAL; – Resultado: -1
```

Actividad desplegable

Una aplicación necesita realizar tres operaciones diferentes sobre valores numéricos. Completa cada situación con la función más adecuada.

situación	función
1. convertir -35 en 35	
2. obtener 18,47 a partir de 18,478 utilizando dos decimales sin redondear	
3. obtener 18,48 a partir de 18,478 utilizando dos decimales	

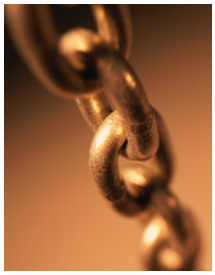
Comprobar | Mostrar retroalimentación

1. **abs.** Devuelve el **valor absoluto** de un número, eliminando su signo negativo.
2. **trunc.** Elimina los decimales que exceden la precisión indicada **sin realizar redondeo**.
3. **round. Redondea** el número al número de decimales indicado, por lo que 18,478 pasa a 18,48 con dos decimales.

5.2.- Funciones de cadena de caracteres.

Las **funciones de cadena de caracteres** permiten **transformar, combinar, buscar o analizar textos**, pudiendo devolver como resultado tanto una cadena como un valor numérico.

Ya verás como es muy común manipular campos de tipo carácter o cadena de caracteres. Como resultado podremos obtener caracteres o números. Estas son las funciones más habituales:



Stockbyte.. *Cadenas de caracteres.* (CC BY-NC-SA)

- **CHR(n)**
  Devuelve el carácter cuyo valor codificado es n.

  Ejemplo:

```
SELECT CHR(81) FROM DUAL; --Resultado: Q
```

- **ASCII(n)**
  Devuelve el valor ASCII de n.

  Ejemplo:

```
SELECT ASCII('O') FROM DUAL; --Resultado: 79
```

- **CONCAT(cad1, cad2)**
  Devuelve las dos cadenas concatenada o unidas. Es equivalente al operador ||

  Ejemplo:

```
SELECT CONCAT('Hola', 'Mundo') FROM DUAL; --Resultado: HolaMundo
```

- **LOWER(cad)**
  Devuelve la cadena cad con todos sus caracteres en minúsculas.

  Ejemplo:

```
SELECT LOWER('En MInúsculAS') FROM DUAL; --Resultado: en minúsculas
```

- **UPPER(cad)**
  Devuelve la cadena cad con todos sus caracteres en mayúsculas.

  Ejemplo:

```
SELECT UPPER('En MAyúsculAS') FROM DUAL; --Resultado: EN MAYÚSCULAS
```

  Esta función y la siguiente son muy utilizadas, y necesarias, al comparar con cadenas cuando hay dudas sobre si todos los caracteres de la cadena a comparar están en mayúsculas, minúsculas o mezcla. Por ejemplo si queremos conocer los datos de la tabla **JUEGOS** cuyo nombre es **AJEDREZ** podemos utilizar cualquiera de las dos sentencias siguientes. Así se seleccionará la fila, tanto si está escrito en la tabla como **AJEDREZ**, **ajedrez o Ajedrez**.

```
SELECT * FROM JUEGOS WHERE UPPER(NOMBRE)='AJEDREZ';
SELECT * FROM JUEGOS WHERE LOWER(NOMBRE)='ajedrez';
```

- **INITCAP(cad)**
  Devuelve la cadena poniendo en **mayúscula la primera letra de cada palabra** y el resto en minúsculas.

  Ejemplo:

```
SELECT INITCAP('hola') FROM DUAL; --Resultado: Hola
```

- **LPAD(cad1, n, cad2)**
  Devuelve cad1 con longitud n, ajustada a la derecha, rellenando por la izquierda con cad2.

  Ejemplo:

```
SELECT LPAD('M', 5, '*') FROM DUAL; --Resultado: ****M
```

- **RPAD(cad1, n, cad2)**
  Devuelve cad1 con longitud n, ajustada a la izquierda, rellenando por la derecha con cad2.

  Ejemplo:

```
SELECT RPAD('M', 5, '*') FROM DUAL; --Resultado: M****
```

- **REPLACE(cad, ant, nue)**
  Devuelve cad en la que cada ocurrencia de la cadena ant ha sido sustituida por la cadena nue.

  Ejemplo:

```
SELECT REPLACE('correo@gmail.es', 'es', 'com') FROM DUAL; --Resultado: correo@gmail.com
```

- **SUBSTR(cad, m, n)**
  Obtiene una subcadena de una cadena. Devuelve la cadena cad compuesta por n caracteres a partir de la posición m.

  Ejemplo:

```
SELECT SUBSTR('1234567', 3, 2) FROM DUAL; --Resultado: 34
```

- **LENGTH(cad)**
  Devuelve la longitud de cad.

  Ejemplo:

```
SELECT LENGTH('hola') FROM DUAL; --Resultado: 4
```

- **TRIM(cad)**
  Elimina los **espacios situados al principio y al final** de la cadena.
  No elimina los espacios dobles del interior, como indica actualmente el contenido.

  Ejemplo:

```
SELECT TRIM(' Hola de nuevo ') FROM DUAL; --Resultado: Hola de nuevo
```

- **LTRIM(cad)**
  Elimina los espacios a la izquierda que posea cad.

  Ejemplo:

```
SELECT LTRIM(' Hola') FROM DUAL; --Resultado: Hola
```

- **RTRIM(cad)**
  Elimina los espacios a la derecha que posea cad.

  Ejemplo:

```
SELECT RTRIM('Hola ') FROM DUAL; --Resultado: Hola
```

INSTR(cad, cadBuscada [, posInicial [, nAparición]])
Obtiene la posición en la que se encuentra la cadena buscada en la cadena inicial cad. Se puede comenzar a buscar desde una posición inicial concreta e incluso indicar el número de aparición de la cadena buscada. Si no encuentra nada devuelve cero.

Ejemplo:

```
SELECT INSTR('usuarios', 'u') FROM DUAL; --Resultado: 1
SELECT INSTR('usuarios', 'u', 2) FROM DUAL; --Resultado: 3
SELECT INSTR('usuarios', 'u', 2, 2) FROM DUAL; --Resultado: 0
```



Autoevaluación

En la siguiente consulta:

```
SELECT LENGTH("Adiós") FROM DUAL;
```

¿Qué ocurriría al ejecutarla en Oracle?

- ○ a. Devolvería 5.
- ○ b. Devolvería 4.
- ○ c. Devolvería un error.

Incorrecta. Aunque la palabra Adiós contiene cinco caracteres, las **comillas dobles** no se utilizan para delimitar cadenas de texto en este caso.

Incorrecta. El problema no está en la cantidad de caracteres, sino en la forma en que se ha escrito el argumento de la función.

Correcta. En Oracle, los literales de texto deben escribirse entre **comillas simples**. La expresión correcta sería

```
SELECT LENGTH('Adiós') FROM DUAL;
```

que devolvería **5**.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

5.3.- Funciones de manejo de fechas.

Las **funciones de manejo de fechas** permiten trabajar con valores temporales, realizar **cálculos entre fechas** y obtener información como la fecha actual, añadir meses o conocer el **tiempo transcurrido** entre dos fechas.

Las **fechas y horas** son datos habituales en una base de datos: pueden representar la emisión de una factura, la llegada de un avión, el registro de un usuario o cualquier otro acontecimiento asociado a un momento determinado.



Stockbyte.. *Calendario.* (CC BY-NC-SA)

Oracle dispone de varios tipos de datos para almacenar información temporal.

*Entre los más utilizados se encuentran **DATE** y **TIMESTAMP**:*

- **DATE.** Almacena **fecha y hora** con precisión de segundos.
- **TIMESTAMP.** Almacena **fecha y hora con mayor precisión**, pudiendo incluir fracciones de segundo.

*Con valores de tipo **DATE** pueden realizarse operaciones aritméticas:*

- **Sumar un número.** Añade ese número de días a la fecha. Los valores decimales representan **fracciones de día**.
- **Restar un número.** Retrocede ese número de días respecto a la fecha indicada.
- **Restar dos valores DATE.** Devuelve el **número de días** transcurridos entre ambas fechas.

En operaciones con **TIMESTAMP**, Oracle puede utilizar **intervalos de tiempo**, por lo que su comportamiento no siempre es idéntico al de DATE.

*En Oracle encontramos, entre otras, las siguientes **funciones de manejo de fechas**:*

- **SYSDATE** Devuelve la fecha y hora actuales.

Ejemplo:

```
SELECT SYSDATE FROM DUAL; --Resultado: 15/08/20
```

- **SYSTIMESTAMP** Devuelve la fecha y hora actuales en formato **TIMESTAMP.**

Ejemplo:

```
SELECT SYSTIMESTAMP FROM DUAL; --Resultado: 15/08/20 11:40:41,969000 +02:00
```

- **ADD_MONTHS(fecha, n)** Añade a la fecha el número de meses indicado con n.

Ejemplo:

```
SELECT ADD_MONTHS('27/07/11', 5) FROM DUAL; --Resultado: 27/12/11
```

- **MONTHS_BETWEEN(fecha1, fecha2)** Devuelve el número de meses que hay entre fecha1 y fecha2.

Ejemplo:

```
SELECT MONTHS_BETWEEN('12/07/11','12/03/11') FROM DUAL; --Resultado: 4
```

- **LAST_DAY(fecha)** Devuelve el último día del mes al que pertenece la fecha. El valor devuelto es tipo **DATE.**

Ejemplo:

```
SELECT LAST_DAY('27/07/11') FROM DUAL; --Resultado: 31/07/11
```

- **NEXT_DAY(fecha, d)** Indica el día que corresponde si añadimos a la fecha el día d. El día devuelto puede ser texto ('Lunes', Martes', ..) o el número del día de la semana (1=lunes, 2=martes, ..) dependiendo de la configuración.

Ejemplo:

```
SELECT NEXT_DAY('31/12/11','LUNES') FROM DUAL; --Resultado: 02/01/12
```

- **EXTRACT(valor FROM fecha)** Extrae un valor de una fecha concreta. El valor puede ser day, month, year, hours, etc.

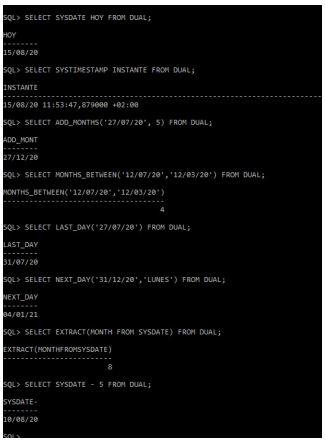
Ejemplo:

```
SELECT EXTRACT(MONTH FROM SYSDATE) FROM DUAL; --Resultado: 8
```

En Oracle: Los operadores aritméticos "+" (más) y "-" (menos) pueden emplearse para las fechas. **Por ejemplo:**

```
SELECT SYSDATE – 5 FROM DUAL; -- Devuelve la fecha correspondiente a 5 días antes de la fecha actual
```

Se pueden emplear estas funciones utilizando como argumento el nombre de un campo de tipo fecha.



Oracle. *Ejecución SQLPlus sentencias fecha* (CC BY-NC-SA)

Autoevaluación

El ejercicio que tienes **es válido, pero necesita pequeños ajustes**. La primera afirmación es demasiado absoluta al decir que Oracle tiene únicamente dos tipos para fechas, y la última debería especificar **DATE** para ser técnicamente precisa.

¿Cuáles de las siguientes afirmaciones sobre el manejo de fechas en Oracle son correctas?

- [ ] a. date y timestamp permiten almacenar información temporal, aunque timestamp puede incluir fracciones de segundo.
- [ ] b. las funciones de fecha pueden recibir como argumento una columna de cualquier tipo de dato sin necesidad de realizar conversiones.
- [ ] c. a un valor date se le puede sumar o restar un número para avanzar o retroceder un determinado número de días.
- [ ] d. la resta entre dos valores date devuelve el número de días existente entre ambas fechas.

Mostrar retroalimentación

Solución

1. Correcto
2. Incorrecto
3. Correcto
4. Correcto

5.4.- Funciones de conversión.

Las **funciones de conversión** permiten transformar explícitamente un dato de un **tipo a otro**, por ejemplo de texto a número, de texto a fecha o de una fecha a una cadena con un formato determinado.

Los SGBD permiten realizar **conversiones entre tipos de datos**. Oracle puede efectuar algunas conversiones de forma implícita, pero es recomendable utilizar **conversiones explícitas** cuando sea posible, ya que resultan más claras y evitan problemas relacionados con los formatos o la configuración regional.

Entre las principales funciones de conversión de Oracle se encuentran:

- **TO_NUMBER(cad, formato).** Convierte una **cadena de caracteres en un valor numérico**. El formato permite indicar cómo debe interpretarse el valor.
- **TO_CHAR(dato, formato).** Convierte un **número, fecha o valor temporal en una cadena de caracteres**, permitiendo controlar su presentación.
- **TO_DATE(cad, formato).** Convierte una **cadena de caracteres en un valor DATE**. El formato especifica cómo debe interpretarse la fecha contenida en el texto.

Formatos numéricos

Símbolo	Significado
9	Posiciones numéricas. Si el número que se quiere visualizar contiene menos dígitos de los que se especifican en el formato, se rellena con blancos.
0	Visualiza ceros por la izquierda hasta completar la longitud del formato especificado.
$	Antepone el signo de dólar al número.
L	Coloca en la posición donde se incluya, el símbolo de la moneda local (se puede configurar en la base de datos mediante el parámetro NSL_CURRENCY)
S	Aparecerá el símbolo del signo.
D	Posición del símbolo decimal, que en español es la coma.
G	Posición del separador de grupo, que en español es el punto.

Formatos de fecha

En las conversiones de fechas pueden utilizarse distintos elementos de formato:

Símbolo	Significado
YY	Año en formato de dos cifras
YYYY	Año en formato de cuatro cifras
MM	Mes en formato de dos cifras
MON	Las tres primeras letras del mes
MONTH	Nombre completo del mes
DY	Día de la semana en tres letras
DAY	Día completo de la semana
DD	Día en formato de dos cifras
D	Día de la semana del 1 al 7
Q	Semestre
WW	Semana del año
AM **PM**	Indicador a.m. Indicador p.m.

Símbolo	Significado
HH12 **HH24**	Hora de 1 a 12 Hora de 0 a 23
MI	Minutos de 0 a 59
SS **SSSS**	Segundos dentro del minuto Segundos dentro desde las 0 horas

Ejemplo:

```
TO_DATE('28/08/2026', 'DD/MM/YYYY')
```

Interpreta la cadena como una **fecha**.

Ejemplo:

```
TO_CHAR(SYSDATE, 'DD/MM/YYYY')
```

Convierte la fecha actual en una **cadena de texto con el formato indicado**.



Autoevaluación

Una aplicación almacena una fecha correctamente como tipo **DATE**, pero necesita mostrarla al usuario con el formato **día/mes/año**.

¿Qué opción permite realizar esta conversión en Oracle?

- ○ a. utilizar to_date para convertir la fecha en texto.
- ○ b. confiar únicamente en una conversión automática.
- ○ c. utilizar to_char indicando el formato 'dd/mm/yyyy'.

Incorrecta. TO_DATE se utiliza para convertir una **cadena de caracteres en un valor DATE**, no para mostrar una fecha como texto.

Incorrecta. Las conversiones automáticas pueden depender de la **configuración** y no permiten controlar de forma explícita el formato mostrado.

Correcta. TO_CHAR permite convertir una fecha en una **cadena de caracteres** y especificar su formato, como **DD/MM/YYYY**.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

5.5.- Otras funciones: NVL y DECODE.

Las funciones **NVL** y **DECODE** permiten tratar **valores NULL** y establecer resultados diferentes según el valor de una expresión, facilitando el control de determinadas situaciones dentro de una consulta.

El valor **NULL** representa un dato **desconocido o inexistente**. Una columna puede contener valores NULL siempre que su definición lo permita; por ejemplo, una columna definida como **NOT NULL** o perteneciente a una **PRIMARY KEY** no puede almacenarlos.

Las operaciones realizadas con un valor **NULL** suelen producir también un resultado NULL. Para tratar estos valores, Oracle proporciona distintas funciones.

- **NVL(valor, expr1).** Si **valor** es NULL, devuelve expr1; en caso contrario, devuelve el propio valor. Ambos argumentos deben tener **tipos compatibles**.

Ejemplo:

```
SELECT NVL(correo, 'No tiene correo') FROM USUARIOS;
```

Si un usuario no tiene correo registrado, se mostrará el texto **No tiene correo** en lugar de NULL.

También podemos devolver distintos resultados dependiendo del valor de una expresión mediante **DECODE**:

- **DECODE(expr, cond1, valor1, cond2, valor2, ..., default).** Compara expr con cada condición. Cuando encuentra una coincidencia, devuelve el **valor asociado**. Si ninguna coincide, devuelve el valor indicado como **default**; si este se omite, devuelve NULL.

Ejemplo:

```
SELECT DECODE(estado, 'A', 'Activo', 'I', 'Inactivo', 'Desconocido')
FROM USUARIOS;
```

En Oracle, **DECODE** continúa estando disponible, aunque para expresiones condicionales también puede utilizarse **CASE**, que ofrece una sintaxis más general y portable entre distintos SGBD.

Autoevaluación

Indica si las siguientes afirmaciones son verdaderas o falsas.

1. si NVL recibe un valor que no es NULL, devuelve ese mismo valor en lugar de la expresión de sustitución.

○ Verdadero ○ Falso

Verdadero

NVL solo utiliza el segundo argumento cuando el primero es **NULL**. Si contiene un valor, devuelve dicho valor.

2. una columna perteneciente a una PRIMARY KEY puede contener NULL y posteriormente sustituirse mediante NVL.

○ Verdadero ○ Falso

Falso

Las columnas que forman una **PRIMARY KEY** no pueden contener valores NULL, por lo que esta situación no sería posible.

3. DECODE puede devolver un valor diferente dependiendo de la coincidencia encontrada y utilizar un resultado por defecto si ninguna se cumple.

○ Verdadero ○ Falso

Verdadero

DECODE compara una expresión con distintos valores y devuelve el resultado asociado a la primera coincidencia; si no existe ninguna, puede devolver el valor **default** indicado.

6.- Consultas de resumen.

Caso práctico

Ada necesita ayuda en otro de los proyectos de la empresa. Quiere obtener información resumida de varias tablas de empleados, como el **salario medio por tipo de empleado** y el **número de trabajadores de cada sucursal**, por lo que pide ayuda a **Juan**.

—No necesito ver todos los salarios uno por uno, sino obtener directamente los resultados generales —comenta **Ada**.

—Entonces tendremos que utilizar **funciones de agregado** y, cuando sea necesario, agrupar los datos —responde **Juan**.



Stockbyte. *Consulta.* (CC BY-NC-SA)

Las **consultas de resumen** utilizan funciones de agregado para realizar cálculos sobre un **conjunto de filas** y obtener resultados como totales, medias, valores máximos, mínimos o recuentos.

En muchas situaciones interesa realizar **cálculos sobre un conjunto de datos** para obtener información global. Por ejemplo, a partir de las calificaciones de un grupo podemos necesitar conocer la **nota máxima**, la **nota media** o el **número de alumnos**.

Para ello se utilizan las **funciones de agregado o agrupamiento**, que procesan un conjunto de filas y devuelven un **resultado resumido**.

Estas funciones pueden utilizarse sobre todos los registros seleccionados o junto con **GROUP BY** para realizar los cálculos sobre diferentes grupos de datos. La cláusula **HAVING** permite posteriormente establecer condiciones sobre esos grupos.

La estructura habitual de muchas funciones de agregado es:

```
FUNCIÓN([ALL | DISTINCT] expresión)
```

Debemos tener en cuenta que:

- **ALL.** Considera todos los valores y es el comportamiento habitual.
- **DISTINCT.** Permite realizar el cálculo considerando únicamente **valores diferentes** cuando la función lo admite.
- Las funciones como **SUM, AVG, MIN y MAX** ignoran normalmente los valores **NULL**.
- **COUNT(expresión)** cuenta únicamente los valores no nulos, mientras que **COUNT(*)** cuenta todas las filas seleccionadas.
- Una consulta puede combinar **funciones de agregado y columnas**, pero las columnas que no formen parte de una función de agregado deberán incluirse normalmente en **GROUP BY**.
- Las funciones de agregado pueden utilizarse en distintos niveles de cálculo cuando la consulta y el SGBD lo permiten.

Ejemplo:

Para obtener el salario medio de todos los empleados:

```
SELECT AVG(SALARIO) FROM EMPLEADOS;
```

El resultado será **un único valor** que resume los salarios procesados por la consulta.

Para saber más

Puedes acceder a este enlace si quieres conocer más sobre este tipo de consultas.

Consultas de resumen.

Reflexiona

Una empresa dispone de una tabla EMPLEADOS con 200 registros. Algunos trabajadores todavía no tienen registrado su teléfono. El responsable necesita saber tanto el **número total de empleados** como cuántos tienen un **teléfono registrado**.

1. ¿Debería utilizarse la misma expresión COUNT para obtener ambos resultados?

2. ¿Qué diferencia existiría entre COUNT(*) y COUNT(telefono)?

Mostrar retroalimentación

No producirían necesariamente el mismo resultado.

COUNT(*) cuenta **todas las filas** seleccionadas, independientemente de que alguna columna contenga valores NULL, por lo que devolvería los 200 empleados.

En cambio, **COUNT(telefono)** solo cuenta las filas en las que **telefono no es NULL**. Por tanto, devolvería únicamente el número de empleados que tienen un teléfono registrado.

Esta diferencia es importante al utilizar funciones de agregado, ya que el tratamiento de los **valores NULL** puede modificar el resultado obtenido.

6.1.- Funciones de agregado: SUM y COUNT.

Las funciones de agregado **SUM** y **COUNT** permiten obtener, respectivamente, la **suma de valores numéricos** y el **número de filas o valores** que cumplen las condiciones de una consulta.

Sumar valores y contar registros son operaciones muy habituales al trabajar con bases de datos. Para ello pueden utilizarse las funciones de agregado **SUM** y **COUNT**.

Función SUM

```
SUM([ALL | DISTINCT] expresión)
```

La función **SUM** devuelve la **suma de los valores** de una expresión numérica. Los valores **NULL** no se tienen en cuenta en el cálculo.

Ejemplo:

```
SELECT SUM(credito) FROM USUARIOS;
```

Esta consulta devuelve la **suma total de los créditos** almacenados.

Función COUNT

```
COUNT([ALL | DISTINCT] expresión)
```

*La función **COUNT** permite realizar distintos tipos de recuento:*

- **COUNT(expresión).** Cuenta los valores **no NULL** obtenidos por la expresión.
- **COUNT(DISTINCT expresión).** Cuenta únicamente los **valores diferentes y no NULL**.
- **COUNT(*).** Cuenta **todas las filas** seleccionadas, aunque alguna de sus columnas contenga NULL.

Ejemplo:

```
SELECT COUNT(nombre) FROM USUARIOS;
```

Cuenta únicamente los registros cuyo campo **nombre no sea NULL**.

Ejemplo:

```
SELECT COUNT(*) FROM USUARIOS;
```

Cuenta el **número total de filas** de la tabla USUARIOS.



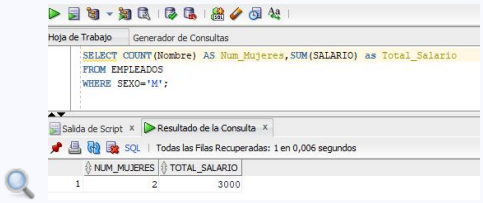
Ejercicio resuelto

Utilizando las tablas y datos de la aplicación **EMPRESA**, se necesita conocer el **número de empleadas** y la **suma total de sus salarios**.

¿Qué consulta permite obtener ambos resultados correctamente?

Mostrar retroalimentación

```sql
SELECT COUNT(Nombre), SUM(SALARIO)
FROM EMPLEADOS
WHERE SEXO = 'M';
```



Ministerio de Educación. *Ejecución sentencia en SQLDeveloper* (CC BY-NC-SA)

6.2.- Funciones de agregado: MIN y MAX.

Las funciones de agregado **MIN** y **MAX** permiten obtener respectivamente el **valor mínimo y máximo** de una expresión sobre el conjunto de filas seleccionado.

Cuando necesitamos localizar el **valor más bajo o más alto** de un conjunto de datos podemos utilizar las funciones **MIN** y **MAX**.

Función MIN

```
MIN([ALL | DISTINCT] expresión)
```

Devuelve el **valor mínimo** de la expresión, sin tener en cuenta los valores **NULL**.

La expresión puede hacer referencia a una columna o a una expresión compatible con la función.

Ejemplo:

```
SELECT MIN(credito) FROM USUARIOS;
```

Esta consulta devuelve el **menor valor** almacenado en la columna credito.

Función MAX

```
MAX([ALL | DISTINCT] expresión)
```

Devuelve el **valor máximo** de la expresión, sin tener en cuenta los valores **NULL**.

Ejemplo:

```
SELECT MAX(credito) FROM USUARIOS;
```

Esta consulta devuelve el **mayor valor** almacenado en la columna credito.

Las funciones **MIN** y **MAX** pueden utilizarse no solo con valores numéricos, sino también con otros tipos de datos comparables, como **fechas** o **cadenas de caracteres**, según el SGBD.



Autoevaluación

Una empresa quiere conocer el **salario más alto** registrado entre sus empleados.
Completa la consulta seleccionando la función adecuada:

SELECT _____(salario) FROM EMPLEADOS;

a. min
b. max
c. count

SELECT [ ](salario) FROM EMPLEADOS;

Enviar

a. **Incorrecta. MIN** devolvería el salario más bajo, no el mayor.
b. **Correcta. MAX** obtiene el **valor más alto** de la columna salario, ignorando los valores NULL.
c. **Incorrecta. COUNT** realiza un **recuento de filas o valores**, pero no permite determinar cuál es el salario más alto.

6.3.- Funciones de agregado: AVG, VAR y STDEV.

Las funciones **AVG, VARIANCE y STDDEV** permiten obtener información estadística de un conjunto de valores, calculando respectivamente su **media, varianza y desviación estándar**.

Podemos necesitar obtener **información estadística** a partir de los datos almacenados en nuestra base de datos. Para ello podemos utilizar funciones de agregado que permiten calcular el promedio, la varianza y la desviación típica.

Función AVG

```
AVG([ALL | DISTINCT] expresión)
```

Devuelve la **media aritmética** de los valores de una expresión. Los valores **NULL** no se tienen en cuenta. Se utiliza principalmente con datos numéricos.

Ejemplo:

```
SELECT AVG(CREDITO) FROM USUARIOS;
```

Esta consulta obtiene la **media de crédito** de los usuarios.

Función VARIANCE

```
VARIANCE([ALL | DISTINCT] expresión)
```

Devuelve la **varianza** de un conjunto de valores numéricos, omitiendo los valores **NULL**. Permite medir cuánto se dispersan los datos respecto a su media.

Función STDDEV

```
STDDEV([ALL | DISTINCT] expresión)
```

Devuelve la **desviación estándar** de un conjunto de valores numéricos, omitiendo los valores **NULL**. Esta medida permite conocer el grado de dispersión de los datos respecto a su media.



Ejercicio resuelto

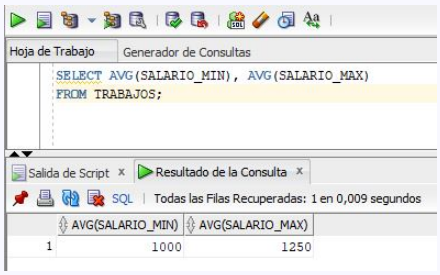
Utilizando las tablas y datos de la aplicación **EMPRESA**, realiza una consulta que obtenga la **media de los salarios mínimos y máximos** de la tabla **TRABAJOS**.

¿Qué consulta permite obtener ambas medias en un único resultado?

Mostrar retroalimentación

```
SELECT AVG(SALARIO_MIN), AVG(SALARIO_MAX) FROM TRABAJOS;
```

La función **AVG** calcula la media de los valores de cada columna, por lo que se obtiene la **media de los salarios mínimos** y la **media de los salarios máximos**.



Elaboración propia. *Ejecución sentencia en SQLDeveloper.* (CC BY-NC-SA)

7.- Agrupamiento de registros.

Caso práctico

Juan ha estado realizando consultas de resumen y ahora quiere obtener información más detallada agrupando los datos según distintos criterios, como el **número de jugadores por provincia** o el **salario medio según el tipo de empleado**.

—Hasta ahora obteníamos un único resultado para todos los registros —comenta **Juan** —. ¿Podemos calcular esos mismos valores por grupos?

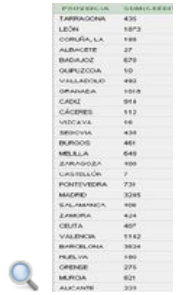
—Sí, podemos agrupar las filas y obtener un resultado para cada grupo —responde **Ada**.



Stockbyte. (Uso educativo nc)

El **agrupamiento de registros** permite reunir filas que comparten un mismo valor y aplicar **funciones de agregado a cada grupo**, obteniendo resultados parciales mediante **GROUP BY** y filtrándolos con **HAVING**.

Las consultas de resumen pueden devolver un **único resultado global**, pero en muchas situaciones interesa calcular **resultados parciales agrupados según una determinada columna**.



Elaboración propia (Uso educativo nc)

Ejemplo:

Podemos obtener el **salario medio de los empleados por ciudad** o calcular la **suma de créditos de los usuarios de cada provincia**.

Para crear estos grupos se utiliza la cláusula **GROUP BY**. Si además queremos establecer condiciones sobre los resultados de los grupos, utilizamos **HAVING**.

La sintaxis general puede incluir:

```
SELECT columna, función_agregado(expresión)
FROM tabla
WHERE condición
GROUP BY columna
HAVING condición_grupo
ORDER BY columna;
```

Debemos diferenciar:

- **WHERE.** Filtra las **filas antes de realizar el agrupamiento**.
- **GROUP BY.** Forma los **grupos** a partir de una o varias columnas.
- **HAVING.** Filtra los **grupos obtenidos**, normalmente mediante condiciones que incluyen funciones de agregado.
- **ORDER BY.** Ordena el **resultado final**.

El orden lógico principal es:

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

Cuando una consulta utiliza **GROUP BY**, las columnas incluidas en SELECT que no formen parte de una función de agregado deben aparecer también en **GROUP BY**.

Ejemplo:

Para obtener la suma de créditos de los usuarios agrupados por provincia:

```
SELECT provincia, SUM(credito)
FROM USUARIOS
GROUP BY provincia;
```

Si únicamente queremos incluir usuarios de **Sevilla o Badajoz**, es preferible filtrar esas filas mediante **WHERE** antes de realizar el agrupamiento:

```sql
SELECT provincia, SUM(credito)
FROM USUARIOS
WHERE UPPER(provincia) IN ('SEVILLA', 'BADAJOZ')
GROUP BY provincia;
```



Ejercicio Resuelto

Utilizando las tablas y datos de la aplicación **EMPRESA**, *resuelve las siguientes consultas:*

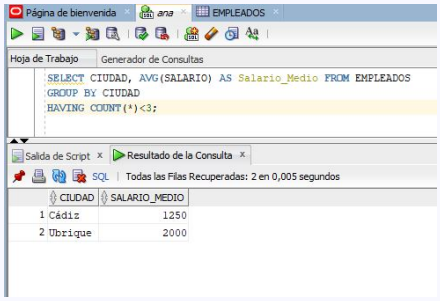
1. Obtener, **agrupado por ciudad**, el **salario medio de los empleados**, mostrando únicamente las ciudades que tengan **menos de 3 empleados**.
2. Obtener de la tabla **HISTORIAL_LABORAL** el **número de empleados distintos que han trabajado en cada departamento**. Si un empleado ha trabajado en distintos periodos en el mismo departamento, deberá contabilizarse **una sola vez**.

¿Qué consultas SQL permiten obtener los resultados solicitados?

Mostrar retroalimentación

```sql
SELECT CIUDAD, AVG(SALARIO) AS Salario_Medio
FROM EMPLEADOS
GROUP BY CIUDAD
HAVING COUNT(*) < 3;
```

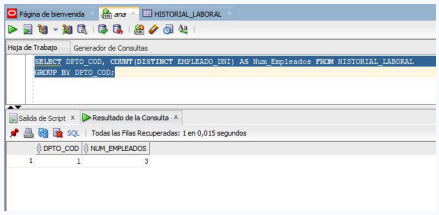
GROUP BY crea un grupo para cada ciudad, **AVG** calcula el salario medio de cada grupo y **HAVING** conserva únicamente las ciudades que tienen menos de tres empleados.



Elaboración propia. *Ejecución en SQLDeveloper.* (CC BY-NC-SA)

```sql
SELECT DPTO_COD, COUNT(DISTINCT EMPLEADO_DNI) AS Num_Empleados
FROM HISTORIAL_LABORAL
GROUP BY DPTO_COD;
```

GROUP BY agrupa los registros por departamento y **COUNT(DISTINCT EMPLEADO_DNI)** cuenta cada empleado una única vez, aunque aparezca en varios periodos dentro del mismo departamento.



Elaboración propia. *Ejecución sentencia en SQLDeveloper.* (CC BY-NC-SA)

8.- Consultas multitablas.

Caso práctico

Hasta ahora **Juan** ha realizado consultas sobre una sola tabla, pero necesita obtener el **historial laboral de los empleados** incluyendo también sus nombres y apellidos.



La información está distribuida entre las tablas **EMPLEADOS** e **HISTORIAL_LABORAL**, relacionadas mediante el identificador del empleado.

—Los datos que necesito están repartidos entre las dos tablas —comenta **Juan**.

—Entonces tendrás que relacionarlas en una misma consulta —responde **Ada**—. Así podrás obtener conjuntamente la información de ambas.

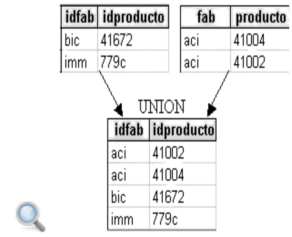
Las **consultas multitabla** permiten recuperar y combinar información almacenada en **dos o más tablas relacionadas**, obteniendo en un mismo resultado datos que se encuentran distribuidos en diferentes partes de la base de datos.

Una característica fundamental de las **bases de datos relacionales** es que la información se distribuye entre varias tablas relacionadas, evitando almacenar innecesariamente los mismos datos en distintos lugares.

Ejemplo:

La tabla **USUARIOS** puede almacenar los datos de cada usuario y la tabla **PARTIDAS** registrar las partidas creadas. Si queremos mostrar conjuntamente el **nombre del usuario** y los datos de sus partidas, necesitaremos consultar ambas tablas y relacionarlas mediante el campo correspondiente.

Las consultas multitabla permiten combinar información mediante diferentes tipos de **JOIN**.



Stockbyte. *Consultas multitabla.* (CC BY-NC-SA)

Entre los más habituales se encuentran:

- **Composición interna (INNER JOIN).** Devuelve las filas que cumplen la condición de relación entre las tablas.
- **Composición externa (OUTER JOIN).** Permite conservar también determinadas filas que no tienen correspondencia en la otra tabla.

El estándar **SQL:1999** incorporó la sintaxis explícita de JOIN, que permite separar con claridad las **tablas que intervienen** de las **condiciones utilizadas para relacionarlas**.

Una sintaxis general puede ser:

```
SELECT tabla1.columna1, tabla2.columna2
FROM tabla1
JOIN tabla2 ON tabla1.columna = tabla2.columna;
```

También pueden utilizarse variantes como **INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN, CROSS JOIN** o **NATURAL JOIN**, según el tipo de combinación que se necesite.

Reflexiona

Una empresa almacena los **datos personales de los empleados** en la tabla EMPLEADOS y los distintos puestos que han ocupado en HISTORIAL_LABORAL. Un desarrollador propone copiar el nombre y los apellidos del empleado en cada registro del historial para no tener que consultar dos tablas.

1. ¿Consideras adecuada esta solución?

2. ¿Qué ventaja ofrece utilizar una consulta multitabla en lugar de repetir los datos del empleado en HISTORIAL_LABORAL?

Mostrar retroalimentación

No sería recomendable **duplicar los datos personales** en cada registro del historial, ya que podrían aparecer **redundancias e inconsistencias** si, por ejemplo, se modifica un apellido en una tabla y no en la otra.

Una **consulta multitabla** permite relacionar EMPLEADOS con HISTORIAL_LABORAL mediante el identificador correspondiente y recuperar conjuntamente la información de ambas tablas. Así, cada dato se mantiene **almacenado una sola vez** y se consulta cuando es necesario.

8.1.- Composiciones internas.

Una **composición interna (INNER JOIN)** combina filas de dos o más tablas y devuelve únicamente aquellas que **cumplen la condición de relación** establecida entre ellas.

Si combinamos dos tablas sin establecer ninguna condición de relación, obtendremos un producto cartesiano: cada fila de una tabla se combinará con todas las filas de la otra.

Si una tabla contiene 10 filas y otra 10, el producto cartesiano generará **100 combinaciones**. Por ello, normalmente necesitaremos indicar qué filas están realmente relacionadas.

Para ello utilizamos una **composición interna o INNER JOIN**, estableciendo una **condición de unión** entre las columnas relacionadas de ambas tablas.

Las reglas principales son:

- **Pueden combinarse varias tablas** en una misma consulta.
- La condición de unión puede utilizar **una o varias parejas de columnas**.
- En **SELECT** pueden aparecer columnas procedentes de cualquiera de las tablas relacionadas.
- Cuando dos tablas tienen columnas con el mismo nombre, debemos indicar su procedencia mediante **tabla.columna** o utilizando **alias**.
- Las columnas utilizadas para relacionar las tablas suelen corresponder a una **clave primaria y una clave foránea**, aunque no es obligatorio que siempre sea así.

La sintaxis recomendada es:

```
SELECT tabla1.columna1, tabla2.columna2
FROM tabla1
INNER JOIN tabla2 ON tabla1.columna = tabla2.columna;
```

También existe la sintaxis tradicional, en la que las tablas aparecen separadas por comas y la condición de unión se incluye en **WHERE**, pero **INNER JOIN** permite distinguir con mayor claridad las condiciones de relación de otros filtros.

Ejemplo:

Para obtener el historial laboral junto con el nombre y los apellidos de cada empleado:

```
SELECT Nombre, Apellido1, Apellido2, Fecha_inicio, Fecha_fin
FROM EMPLEADOS
INNER JOIN HISTORIAL_LABORAL
ON HISTORIAL_LABORAL.Empleado_DNI = EMPLEADOS.DNI;
```

Si además necesitamos obtener el **nombre del departamento**, será necesario incorporar también la tabla DEPARTAMENTOS:

```
SELECT Nombre_Dpto, Nombre, Apellido1, Apellido2
FROM EMPLEADOS
INNER JOIN HISTORIAL_LABORAL
ON EMPLEADOS.DNI = HISTORIAL_LABORAL.EMPLEADO_DNI
INNER JOIN DEPARTAMENTOS
ON HISTORIAL_LABORAL.DPTO_COD = DEPARTAMENTOS.DPTO_COD;
```

Una tabla también puede relacionarse **consigo misma**. En estos casos deben utilizarse **alias diferentes** para distinguir los distintos papeles que desempeña cada aparición de la tabla.



Ejercicio resuelto

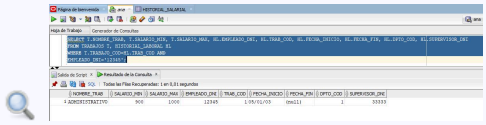
Utilizando las tablas y datos de la aplicación **EMPRESA**, realiza una consulta que obtenga el **nombre de los empleados** junto con los **salarios que han tenido**, utilizando para ello las tablas **EMPLEADOS** e **HISTORIAL_SALARIAL**.

¿Qué consulta permite relacionar ambas tablas y obtener el nombre de cada empleado junto con su historial salarial?

Mostrar retroalimentación

La consulta debe relacionar **EMPLEADOS** e **HISTORIAL_SALARIAL** mediante el **DNI del empleado**, que es el dato común entre ambas tablas.

De esta forma, la **composición interna** devuelve únicamente las filas que tienen correspondencia en las dos tablas y permite mostrar conjuntamente el **nombre del empleado** y sus **salarios registrados**.



Elaboración propia. *Ejecución en SQLDeveloper de la sentencia* (CC BY-NC-SA)

8.2.- Composiciones externas.

Las **composiciones externas (OUTER JOIN)** permiten combinar tablas conservando también las **filas que no tienen correspondencia** en la otra tabla, completando con valores NULL los datos que no existen.

En ocasiones interesa recuperar determinadas filas aunque **no tengan una fila relacionada** en la otra tabla.



Stockbyte (Uso educativo nc)

Ejemplo:

Supongamos que disponemos de las tablas **EMPLEADOS** y **DEPARTAMENTOS** y se ha creado recientemente un departamento al que todavía no se ha asignado ningún empleado. Si queremos obtener un listado de **todos los departamentos**, también deberá aparecer ese nuevo departamento aunque no tenga empleados asociados.

*Para estos casos se utilizan las **composiciones externas**:*

- **LEFT OUTER JOIN.** Conserva todas las filas de la tabla situada a la izquierda.
- **RIGHT OUTER JOIN.** Conserva todas las filas de la tabla situada a la derecha.
- **FULL OUTER JOIN.** Conserva todas las filas de ambas tablas, tengan o no correspondencia.

Cuando una fila no tiene correspondencia, las columnas procedentes de la otra tabla aparecen con valores **NULL**.

La sintaxis recomendada es:

```
SELECT columnas
FROM tabla1
LEFT OUTER JOIN tabla2 ON tabla1.columna = tabla2.columna;
```

Oracle también admite la sintaxis tradicional mediante el operador **(+)** dentro de la cláusula WHERE. Sin embargo, actualmente recomienda utilizar la sintaxis **OUTER JOIN** en la cláusula FROM.

Ejercicio resuelto

Se necesita obtener un listado con los **nombres de todos los departamentos** y los datos personales de sus jefes. Deben aparecer también aquellos departamentos que **todavía no tengan ningún jefe asignado**.

¿Qué consulta permite conservar todos los departamentos aunque no exista un empleado asociado como jefe?

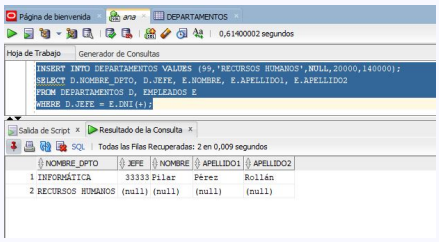
Mostrar retroalimentación

La consulta puede realizarse mediante una **composición externa**:

```
SELECT D.NOMBRE_DPTO, D.JEFE, E.NOMBRE, E.APELLIDO1, E.APELLIDO2
FROM DEPARTAMENTOS D, EMPLEADOS E
WHERE D.JEFE = E.DNI(+);
```

El **LEFT OUTER JOIN** conserva todos los registros de **DEPARTAMENTOS**. Cuando un departamento no tiene jefe asociado, sus datos siguen apareciendo y las columnas correspondientes a EMPLEADOS toman el valor **NULL**.

La consulta de la captura utiliza la sintaxis tradicional de Oracle con **(+)**, que sigue siendo válida y produce el mismo tipo de composición externa, aunque la sintaxis con **OUTER JOIN** es la recomendada actualmente por Oracle.



Elaboración propia. *Ejecución en SQLDeveloper* (CC BY-NC-SA)

8.3.- Composiciones en la versión SQL:1999.

El estándar **SQL:1999** incorporó una sintaxis explícita para realizar **composiciones entre tablas**, permitiendo expresar con mayor claridad qué tablas se relacionan y mediante qué condiciones.



Stockbyte (Uso educativo nc)

La sintaxis introducida en **SQL:1999** permite realizar distintos tipos de composiciones mediante la cláusula **JOIN**. Esta forma separa las **condiciones de unión entre tablas** de otros criterios de filtrado y facilita la lectura de las consultas.

La sintaxis general contempla distintas posibilidades:

- **CROSS JOIN.** Genera el **producto cartesiano**, combinando cada fila de una tabla con todas las filas de la otra.
- **NATURAL JOIN.** Relaciona automáticamente las columnas que tienen el **mismo nombre** en ambas tablas. Debe utilizarse con precaución, ya que puede incluir columnas coincidentes que no queramos usar para la relación.
- **JOIN ... USING.** Permite indicar qué **columnas con el mismo nombre** deben utilizarse para relacionar las tablas.
- **JOIN ... ON.** Permite establecer de forma explícita la **condición de unión**, incluso cuando las columnas relacionadas tienen nombres diferentes.
- **LEFT OUTER JOIN.** Devuelve todas las filas de la tabla situada a la **izquierda** y las coincidencias de la derecha.
- **RIGHT OUTER JOIN.** Devuelve todas las filas de la tabla situada a la **derecha** y las coincidencias de la izquierda.
- **FULL OUTER JOIN.** Devuelve las filas de **ambas tablas**, tengan o no correspondencia.

Ejemplo:

Para obtener el historial laboral de los empleados incluyendo sus nombres, apellidos y las fechas de inicio y fin:

```
SELECT E.Nombre, E.Apellido1, E.Apellido2, H.Fecha_inicio, H.Fecha_fin
FROM EMPLEADOS E JOIN HISTORIAL_LABORAL H
ON H.Empleado_DNI = E.DNI;
```

En este caso, **JOIN ... ON** establece de forma explícita la relación entre ambas tablas.

Ejemplo:

Para obtener todos los departamentos y los datos de sus jefes, incluyendo aquellos que todavía no tengan ninguno asignado:

```
SELECT D.NOMBRE_DPTO, D.JEFE, E.NOMBRE, E.APELLIDO1, E.APELLIDO2
FROM DEPARTAMENTOS D LEFT OUTER JOIN EMPLEADOS E
ON D.JEFE = E.DNI;
```

El **LEFT OUTER JOIN** conserva todos los departamentos aunque no exista un empleado relacionado.

Para saber más

En MySQL también se utilizan las composiciones, aquí puedes verlo:

Composiciones.

Actividad desplegable

Selecciona el tipo de composición más adecuado para cada situación.

Situación	Respuesta
1. combinar dos tablas indicando expresamente que empleado_dni debe coincidir con dni	

Situación	Respuesta
2. obtener todos los departamentos aunque algunos no tengan jefe, situando departamentos a la izquierda	
3. obtener todas las combinaciones posibles entre las filas de dos tablas	

Comprobar | Mostrar retroalimentación

1. **join ... on.** Permite establecer de forma explícita la **condición utilizada para relacionar las tablas**, incluso si las columnas tienen nombres diferentes.
2. **left outer join.** Conserva todas las filas de la **tabla situada a la izquierda**, aunque no exista correspondencia en la otra.
3. **cross join.** Genera el **producto cartesiano**, combinando cada fila de una tabla con todas las filas de la otra.

9.- Consultas multiselección: Unión, Intersección y diferencia de consultas.

Caso práctico

Ana comenta a **Carlos** que ya tienen prácticamente terminado el trabajo, pero quiere enseñarle otros tipos de consultas que pueden resultar útiles para **combinar resultados**.

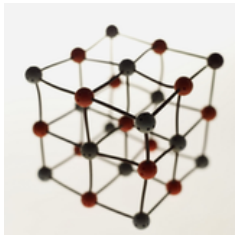


—¿También podemos trabajar con los resultados de varias consultas como si fueran conjuntos? —pregunta **Carlos**.

—Exactamente —responde **Ana**—. Podemos **unirlos, buscar los elementos comunes o calcular sus diferencias**.

Las **consultas multiselección** permiten combinar los resultados de varias sentencias SELECT mediante operaciones de **unión, intersección o diferencia**, tratándolos como conjuntos de filas.

Una consulta **SELECT** devuelve un conjunto de filas. SQL permite combinar los resultados de varias consultas mediante **operadores de conjuntos**, siempre que sus resultados sean compatibles.



Stockbyte (Uso educativo nc)

Las principales operaciones son:

- **UNION.** Combina las filas obtenidas por dos consultas y **elimina los duplicados**.
- **INTERSECT.** Devuelve únicamente las filas que aparecen en **ambos resultados**, eliminando duplicados.
- **MINUS.** En Oracle, devuelve las filas que aparecen en el **primer SELECT pero no en el segundo**, eliminando duplicados. Las versiones actuales de Oracle admiten también **EXCEPT** con el mismo significado.

También existe **UNION ALL**, que combina los resultados pero **conserva las filas duplicadas**.

Para utilizar estos operadores, las consultas combinadas deben devolver:

- El **mismo número de columnas**.
- Las columnas en el **mismo orden**.
- **Tipos de datos compatibles** en las posiciones correspondientes.

Cuando se combinan varias operaciones de conjuntos, pueden utilizarse **paréntesis** para indicar claramente el orden en que deben evaluarse.

Unión

Ejemplo:

Obtener los nombres y ciudades de los proveedores y clientes de Alemania:

```
SELECT NombreCia, Ciudad FROM PROVEEDORES WHERE Pais = 'Alemania'
UNION
SELECT NombreCia, Ciudad FROM CLIENTES WHERE Pais = 'Alemania';
```

UNION reúne los resultados y elimina las filas que estuvieran repetidas.

Intersección

Ejemplo:

Una academia almacena en tablas diferentes los alumnos matriculados en inglés, francés y portugués. Se quieren obtener los alumnos matriculados en los **tres idiomas**:

```
SELECT nombre, domicilio FROM ingles
INTERSECT
SELECT nombre, domicilio FROM frances
```

```
INTERSECT
SELECT nombre, domicilio FROM portugues;
```

INTERSECT conserva únicamente las filas presentes en **todos los resultados combinados**.

Diferencia

Ejemplo:

Obtener los alumnos matriculados en inglés que **no están matriculados en portugués**:

```
SELECT nombre, domicilio FROM ingles
MINUS
SELECT nombre, domicilio FROM portugues;
```

En Oracle, **MINUS** devuelve las filas del primer resultado que no aparecen en el segundo. En versiones actuales también puede utilizarse **EXCEPT** como equivalente.



Autoevaluación

¿Cuáles de las siguientes afirmaciones sobre las consultas multiselección son correctas?

- [ ] a. union combina las filas de un primer select con las de otro select, eliminando las filas duplicadas.
- [ ] b. minus devuelve las filas que aparecen en el primer select pero no en el segundo.
- [ ] c. intersect devuelve las filas que aparecen en ambos resultados.
- [ ] d. en union, intersect y minus, los select deben devolver el mismo número de columnas, pero sus tipos de datos no necesitan ser compatibles.

Mostrar retroalimentación

Solución

1. Correcto
2. Correcto
3. Correcto
4. Incorrecto

10.- Subconsultas.

Caso práctico

—¿Es posible hacer una consulta dentro de otra consulta? —pregunta **Carlos**.

Carlos se ha dado cuenta de que, en ocasiones, necesitará filtrar datos utilizando un **resultado que todavía desconoce**.

—Sí —responde **Ana**—. Podemos obtener primero ese resultado mediante una **subconsulta** y utilizarlo después en la consulta principal.



Stockbyte (Uso educativo nc)

Una **subconsulta** es una sentencia SELECT incluida dentro de otra consulta, cuyo resultado se utiliza para **filtrar, comparar o calcular datos** en la consulta principal.

En algunas situaciones necesitamos utilizar dentro de una consulta el **resultado obtenido por otra sentencia SELECT**. Esta consulta interna recibe el nombre de **subconsulta** o **subselect**.

Una subconsulta se escribe normalmente **entre paréntesis** y puede utilizarse en diferentes partes de una sentencia, como **WHERE, HAVING, FROM** o incluso **SELECT**, dependiendo del resultado que produzca.

Su estructura básica puede ser:

```
SELECT listaExpresiones
FROM tabla
WHERE expresión OPERADOR
(SELECT expresión
FROM tabla);
```

Subconsultas de una sola fila

Cuando la subconsulta devuelve **un único valor**, puede utilizarse con operadores de comparación como **=, <>, !=, <, >, <= o >=**.

Ejemplo:

```
SELECT Nombre, salario
FROM EMPLEADOS
WHERE salario <
(SELECT salario FROM EMPLEADOS WHERE Nombre = 'Ana');
```

Esta consulta obtiene los empleados cuyo salario es **inferior al de Ana**. Si la subconsulta devolviera varios salarios, el operador **<** no podría realizar una comparación escalar y se produciría un error.

Los valores comparados deben tener **tipos de datos compatibles**.

Subconsultas de varias filas

*Cuando la subconsulta puede devolver **varias filas**, deben utilizarse operadores adecuados:*

- **ANY.** La comparación se cumple si es verdadera para **al menos uno** de los valores devueltos.
- **ALL.** La comparación debe cumplirse respecto a **todos** los valores devueltos.
- **IN.** Comprueba si un valor se encuentra entre los resultados de la subconsulta.
- **NOT IN.** Comprueba si un valor no se encuentra entre los resultados de la subconsulta.

Ejemplo:

Para obtener el empleado con el salario más bajo:

```sql
SELECT nombre, salario
FROM EMPLEADOS
WHERE salario <= ALL (SELECT salario FROM EMPLEADOS);
```

*También puede resolverse utilizando la función **MIN**:*

```sql
SELECT nombre, salario
FROM EMPLEADOS
WHERE salario = (SELECT MIN(salario) FROM EMPLEADOS);
```



Autoevaluación

Relaciona cada instrucción con su función:

instrucción	relación	función
any		1. comprueba si un valor no aparece entre los resultados de la subconsulta.
all		2. la comparación debe cumplirse con todos los valores devueltos por la subconsulta.
in		3. la comparación se cumple con al menos uno de los valores devueltos por la subconsulta.
not in		4. comprueba si un valor aparece entre los resultados de la subconsulta.

Enviar

1. **ANY → 3.** La comparación se cumple cuando resulta verdadera para **al menos uno** de los valores devueltos por la subconsulta.
2. **ALL → 2.** La comparación debe cumplirse respecto a **todos los valores** obtenidos por la subconsulta.
3. **IN → 4.** Comprueba si un valor está incluido entre los **resultados de la subconsulta**.
4. **NOT IN → 1.** Comprueba si un valor **no se encuentra** entre los resultados devueltos por la subconsulta.

11.- Optimización de consultas.

La **optimización de consultas** busca reducir el tiempo de respuesta y el consumo de recursos mediante un buen **diseño de la base de datos**, consultas eficientes, índices adecuados y el análisis del **plan de ejecución**.

Cuando aumenta el **volumen de datos**, el número de usuarios o la complejidad de las consultas, pueden aparecer problemas de rendimiento. Por ello, es importante diseñar correctamente la base de datos y revisar cómo se ejecutan las consultas.



humanlevel.com. *Optimización de consultas* (CC BY-NC-SA <http://creativecommons.org/licenses/?lang=es>)

Algunas prácticas que pueden mejorar el rendimiento son:

- **Crear índices** en columnas utilizadas frecuentemente para búsquedas, filtros o relaciones, siempre que aporten una mejora real.
- **Seleccionar únicamente las columnas necesarias**, evitando recuperar información que no va a utilizarse.
- Utilizar **WHERE** para filtrar cuanto antes las filas que no sean necesarias.
- Evitar aplicar **funciones o transformaciones innecesarias** sobre columnas utilizadas para búsquedas, ya que pueden dificultar el aprovechamiento de determinados índices.
- Utilizar **DISTINCT** únicamente cuando sea necesario eliminar duplicados.
- Utilizar **JOIN** con condiciones de unión claras y evitar **productos cartesianos accidentales**.
- En tablas de gran tamaño, valorar técnicas como el **particionamiento** cuando las características de los datos y las consultas lo justifiquen.
- Analizar el **plan de ejecución** para comprobar cómo accede el SGBD a los datos y detectar operaciones costosas.

No existe una regla que haga que una consulta sea siempre más rápida. La optimización debe basarse en **mediciones reales**, teniendo en cuenta el volumen de datos, los índices disponibles, las estadísticas y el tipo de consulta.

Ejercicio Resuelto

Una empresa dispone de una base de datos con una tabla de contactos que inicialmente contenía pocos registros. Con el tiempo ha superado el **millón de filas** y ahora es utilizada simultáneamente por numerosos empleados. Las búsquedas por **nombre** se han vuelto lentas y los tiempos de respuesta han aumentado considerablemente.

¿Qué medidas podrían estudiarse para mejorar el rendimiento de las consultas sin modificar innecesariamente los datos almacenados?

Mostrar retroalimentación

Antes de realizar cambios debe analizarse cómo se están ejecutando las consultas mediante el **plan de ejecución**.

Si las búsquedas por nombre son frecuentes, puede estudiarse la creación de un **índice adecuado** sobre la columna utilizada, comprobando después si realmente mejora el rendimiento. También conviene revisar que las consultas recuperen únicamente las **columnas y filas necesarias** y evitar operaciones que impidan aprovechar los índices disponibles.

La optimización debe realizarse a partir de **mediciones y pruebas**, ya que una técnica beneficiosa para una consulta puede no serlo para otra.

Anexo I.- Base de datos de ejemplo (Juegos online).

Esta base de datos de ejemplo representa un sistema de **juegos online** y permite practicar consultas SQL sobre **usuarios, juegos y partidas**, utilizando tablas relacionadas mediante claves primarias y foráneas.

Enunciado.

Un sitio de **juegos online** necesita una base de datos para gestionar sus **usuarios, juegos y partidas**.

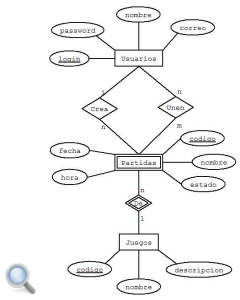
Cuando un usuario accede al sistema debe identificarse mediante su **login y contraseña**. Si todavía no dispone de una cuenta, se registrarán sus datos personales y de acceso.

De cada **juego** se almacena un código identificador, su nombre y una descripción.

Los usuarios pueden crear **partidas** de los juegos disponibles o unirse a partidas ya existentes. De cada partida se almacena un código, nombre, estado, juego asociado, fecha y hora de creación y el usuario que la ha creado.

Cada partida pertenece a **un único juego**, mientras que un juego puede tener **varias partidas**. Además, una partida puede contar con varios usuarios y un usuario puede participar en varias partidas.

Diagrama E-R.



Ministerio de Educación. (Uso Educativo nc)

Estructura de Tablas

USUARIOS	PARTIDAS	JUEGOS	UNEN
login VARCHAR2(15)	codigo VARCHAR2(15)	codigo VARCHAR2(15)	codigo_partida VARCHAR2(15)
password VARCHAR2(9)	nombre VARCHAR2(25)	nombre VARCHAR2(25)	codigo_usuario VARCHAR2(15)
nombre VARCHAR2(25)	estado VARCHAR2(1)	descripcion VARCHAR2(200)	
apellidos VARCHAR2(30)	cod_juego VARCHAR2(15)		
direccion VARCHAR2(30)	fecha_inicio_partida DATE		
cp VARCHAR2(5)	hora_inicio_partida TIMESTAMP		
localidad VARCHAR2(25)	cod_creador_partida VARCHAR2(15)		
provincia VARCHAR2(25)			
pais VARCHAR2(15)			
f_nacimiento DATE			
f_ingreso DATE			
correo VARCHAR2(25)			
credito NUMBER			
sexo VARCHAR2(1)			

Importante: las contraseñas utilizadas son únicamente **datos ficticios para prácticas**. En una aplicación real no deben almacenarse directamente, sino mediante mecanismos seguros de **hash de contraseñas**.

Sentencias de creación de tablas

A continuación se presentan unas posibles sentencias SQL de creación de las tablas del ejercicio.

TABLA USUARIOS

```
CREATE TABLE USUARIOS (
         login         VARCHAR2(15) PRIMARY KEY NOT NULL,
         password      VARCHAR2(9) NOT NULL,
         nombre        VARCHAR2(25) NOT NULL,
         apellidos     VARCHAR2(30) NOT NULL,
         direccion     VARCHAR2(30) NOT NULL,
         cp            VARCHAR2(5) NOT NULL,
         localidad     VARCHAR2(25) NOT NULL,
         provincia     VARCHAR2(25) NOT NULL,
         pais          VARCHAR2(15) NOT NULL,
         f_nac      DATE,
         f_ing      DATE DEFAULT (sysdate),
         correo     VARCHAR2(25) NOT NULL,
         credito    NUMBER,
         sexo       VARCHAR2(1));
```

TABLA JUEGOS

```
CREATE TABLE JUEGOS (
    codigo VARCHAR2(15) PRIMARY KEY NOT NULL,
    nombre VARCHAR2(15) NOT NULL,
    descripcion VARCHAR2(200) NOT NULL
);
```

TABLA PARTIDAS

```
CREATE TABLE PARTIDAS (
    codigo VARCHAR2(15) PRIMARY KEY NOT NULL,
    nombre VARCHAR2(25) NOT NULL,
    estado VARCHAR2(1) NOT NULL,
    cod_juego VARCHAR2(15) NOT NULL,
    CONSTRAINT CA_cod_juego REFERENCES JUEGOS(codigo),
    fecha_inicio DATE,
    hora_inicio TIMESTAMP,
    cod_creador VARCHAR2(15),
    CONSTRAINT CA_cod_creador REFERENCES USUARIOS(login)
);
```

TABLA UNEN

```
CREATE TABLE UNEN (
    codigo_partida VARCHAR2(15) NOT NULL,
    CONSTRAINT CA_codigo_partida REFERENCES PARTIDAS(codigo),
    codigo_usuario VARCHAR2(15) NOT NULL,
    CONSTRAINT CA_codigo_usuario REFERENCES USUARIOS(login),
    CONSTRAINT PK_UNEN PRIMARY KEY (codigo_partida, codigo_usuario)
);
```

Ejemplos de datos.

TABLA USUARIOS(Campos desde Login a Código Postal):

LOGIN	PASSWORD	NOMBRE	APELLIDOS	DIRECCION	CP
anamat56	JD9U6?	ANA M.	MATA VARGAS	GARCILASO DE LA VEGA	08924
alecam89	5;5@PK	ALEJANDRO EMILIO	CAMINO LAZARO	PEDRO AGUADO BLEYE	34004
verbad64	MP49HF	VERONICA	BADIOLA PICAZO	BARRANCO GUINIGUADA	35015

LOGIN	PASSWORD	NOMBRE	APELLIDOS	DIRECCION	CP
conmar76	O1<N9U	CONSUELO	MARTINEZ RODRIGUEZ	ROSA	04002
encpay57	FYC3L5	ENCARNACIÓN	PAYO MORALES	MULLER, AVINGUDA	43007
mandia79	00JRIH	MANUELA	DIAZ COLAS	214 (GENOVA)	07015
alibar52	IER8S	ALICIA MARIA	BARRANCO CALLIZO	HECTOR VILLALOBOS	29014
adofid63	;82=MH	ADOLFO	FIDALGO DIEZ	FORCALL	12006
jesdie98	X565ZS	JESUS	DIEZ GIL	TABAIBAL	35213
pedsan70	T?5=J@	PEDRO	SANCHEZ GUIL	PINTOR ZULOAGA	03013
diahue96	LSQZMC	DIANA	HUERTA VALIOS	JOAQUIN SALAS	39011
robrod74	<LQMLP	ROBERTO	RODRIGUEZ PARMO	CASTILLO HIDALGO	51002
milgar78	SF=UZ8	MILAGROSA	GARCIA ELVIRA	PEDRALBA	28037
frabar93	19JZ7@	FRANCISCA	BARRANCO RODRIGUEZ	BALSAS, LAS	26006
migarc93	AAFLTW	MIGUEL ANGEL	ARCOS ALONSO	ISAAC ALBENIZ	04008

TABLA USUARIOS (Campos desde Localidad a Sexo):

LOCALIDAD	PROVINCIA	PAIS	F_NACIMIENTO	F_INGRESO	CORREO	CREDITO	SEXO
SANTA COLOMA DE GRAMANET	BARCELONA	ESPAÑA	08/25/1974	10/10/2007	anamat56@hotmail.com	213	M
PALENCIA	PALENCIA	ESPAÑA	05/03/1976	10/15/2010	alecam89@hotmail.com	169	H
PALMAS GRAN CANARIA,LAS	PALMAS (LAS)	ESPAÑA	01/28/1984	10/23/2010	verbad64@hotmail.com	437	M
ALMERÍA	ALMERÍA	ESPAÑA	08/09/1978	03/25/2007	conmar76@yahoo.com	393	M
TARRAGONA	TARRAGONA	ESPAÑA	05/04/1993	01/06/2010	encpay57@yahoo.com	318	M
PALMA DE MALLORCA	BALEARES	ESPAÑA	07/14/1979	07/16/2008	mandia79@hotmail.com	255	M
MÁLAGA	MÁLAGA	ESPAÑA	08/21/1993	09/19/2010	alibar52@hotmail.com	486	M
CASTELLÓN DE LA PLANA	CASTELLÓN	ESPAÑA	08/11/1981	03/02/2008	adofid63@gmail.com	154	H
TELDE	PALMAS (LAS)	ESPAÑA	10/23/1981	09/13/2009	jesdie98@gmail.com	152	H
ALACANT/ALICANTE	ALICANTE	ESPAÑA	12/01/1983	06/15/2008	pedsan70@yahoo.com	21	H
SANTANDER	CANTABRIA	ESPAÑA	04/25/1984	07/31/2009	diahue96@yahoo.com	395	M
CEUTA	CEUTA	ESPAÑA	06/28/1978	03/16/2009	robrod74@gmail.com	486	H
MADRID	MADRID	ESPAÑA	04/12/1983	05/15/2008	milgar78@gmail.com	330	M
LOGROÑO	RIOJA (LA)	ESPAÑA	09/21/1986	02/16/2008	frabar93@gmail.com	75	M
ALMERÍA	ALMERÍA	ESPAÑA	03/01/1991	06/16/2010	migarc93@hotmail.com	23	H

TABLA PARTIDAS:

CODIGO	NOMBRE	ESTADO	COD_JUEGO	FECHA	HORA	COD_CREADOR
1	Billar_migarc93_18/7	1	12	07/18/2011	00:47:40	migarc93
2	Chinchón_mandia79_2/10	1	6	10/02/2011	01:47:40	mandia79

CODIGO	NOMBRE	ESTADO	COD_JUEGO	FECHA	HORA	COD_CREADOR
3	Canasta_alibar52_26/2	0	8	02/26/2011	08:57:33	alibar52
4	Damas_verbad64_16/3	1	4	03/16/2011	00:53:00	verbad64
5	Chinchón_alibar52_9/9	1	6	09/09/2011	09:10:22	alibar52
6	Oca_pedsan70_21/12	0	2	12/21/2011	18:53:17	pedsan70
7	Canasta_encpay57_18/2	0	8	02/18/2011	09:41:02	encpay57
8	Pocha_adofid63_26/10	1	10	10/26/2011	02:23:43	adofid63
9	Damas_diahue96_25/6	1	4	06/25/2011	18:11:14	diahue96
10	Parchís_encpay57_31/7	1	1	07/31/2011	21:21:36	encpay57

TABLA JUEGOS:

CODIGO	NOMBRE	DESCRIPCION
1	Parchís	El parchís es un juego de mesa derivado del pachisi y similar al ludo y al parcheesi.
2	Oca	El juego de la oca es un juego de mesa para dos o más jugadores.
3	Ajedrez	El ajedrez es un juego entre dos personas, cada una de las cuales dispone de 16 piezas móviles que se colocan sobre un tablero dividido en 64 escaques.
4	Damas	Las damas es un juego de mesa para dos contrincantes.
5	Poker	El póquer es un juego de cartas de los llamados de apuestas.
6	Chinchón	El chinchón es un juego de naipes de 2 a 8 jugadores.
7	Mus	El mus es un juego de naipes, originario de Navarra, que se encuentra muy extendido por toda España.
8	Canasta	La canasta o rummy-canasta es un juego de naipes, variante del rummy.
9	Dominó	El dominó es un juego de mesa en el que se emplean unas fichas rectangulares.
10	Pocha	La pocha es un juego de cartas que se juega con la baraja española.
11	Backgammon	Cada jugador tiene quince fichas que va moviendo entre veinticuatro triángulos según el resultado de sus dos dados.
12	Billar	El billar es un deporte de precisión que se practica impulsando con un taco un número variable de bolas.

TABLA UNEN:

CODIGO_PARTIDA	CODIGO_USUARIO
4	anamat56
3	alecam89
6	alecam89
2	conmar76
2	encpay57
2	mandia79
4	alibar52
3	adofid63
5	jesdie98

CODIGO_PARTIDA	CODIGO_USUARIO
8	pedsan70
6	diahue96
4	robrod74
5	milgar78
4	frabar93
5	encpay57

Inserción de datos

Las siguientes sentencias **INSERT** permiten cargar los datos utilizados en los ejemplos y ejercicios.

Formato de las fechas

```
ALTER SESSION SET NLS_DATE_FORMAT='MM/DD/YYYY';
```

TABLA USUARIOS

```
INSERT INTO USUARIOS VALUES(
    'anamat56','JD9U6?','ANA M.','MATA VARGAS',
    'GARCILASO DE LA VEGA','08924','SANTA COLOMA DE GRAMANET',
    'BARCELONA','ESPAÑA','08/25/1974','10/10/2007',
    'anamat56@hotmail.com',213,'M'
);

INSERT INTO USUARIOS VALUES(
    'alecam89','5;5@PK','ALEJANDRO EMILIO','CAMINO LAZARO',
    'PEDRO AGUADO BLEYE','34004','PALENCIA','PALENCIA','ESPAÑA',
    '05/03/1976','10/15/2010','alecam89@hotmail.com',169,'H'
);

INSERT INTO USUARIOS VALUES(
    'verbad64','MP49HF','VERONICA','BADIOLA PICAZO',
    'BARRANCO GUINIGUADA','35015','PALMAS GRAN CANARIA,LAS',
    'PALMAS (LAS)','ESPAÑA','01/28/1984','10/23/2010',
    'verbad64@hotmail.com',437,'M'
);

INSERT INTO USUARIOS VALUES(
    'conmar76','O1<N9U','CONSUELO','MARTINEZ RODRIGUEZ',
    'ROSA','04002','ALMERÍA','ALMERÍA','ESPAÑA',
    '08/09/1978','03/25/2007','conmar76@yahoo.com',393,'M'
);

INSERT INTO USUARIOS VALUES(
    'encpay57','FYC3L5','ENCARNACIÓN','PAYO MORALES',
    'MULLER,AVINGUDA','43007','TARRAGONA','TARRAGONA','ESPAÑA',
    '05/04/1993','01/06/2010','encpay57@yahoo.com',318,'M'
);

INSERT INTO USUARIOS VALUES(
    'mandia79','00JRIH','MANUELA','DIAZ COLAS',
    '214 (GENOVA)','07015','PALMA DE MALLORCA','BALEARES','ESPAÑA',
    '07/14/1979','07/16/2008','mandia79@hotmail.com',255,'M'
);

INSERT INTO USUARIOS VALUES(
    'alibar52','IER8S','ALICIA MARIA','BARRANCO CALLIZO',
    'HECTOR VILLALOBOS','29014','MÁLAGA','MÁLAGA','ESPAÑA',
    '08/21/1993','09/19/2010','alibar52@hotmail.com',486,'M'
);

INSERT INTO USUARIOS VALUES(
    'adofid63',';82=MH','ADOLFO','FIDALGO DIEZ',
    'FORCALL','12006','CASTELLÓN DE LA PLANA','CASTELLÓN','ESPAÑA',
    '08/11/1981','03/02/2008','adofid63@gmail.com',154,'H'
);
```

```
INSERT INTO USUARIOS VALUES(
    'jesdie98','X565ZS','JESUS','DIEZ GIL',
    'TABAIBAL','35213','TELDE','PALMAS (LAS)','ESPAÑA',
    '10/23/1981','09/13/2009','jesdie98@gmail.com',152,'H'
);

INSERT INTO USUARIOS VALUES(
    'pedsan70','T?5=J@','PEDRO','SANCHEZ GUIL',
    'PINTOR ZULOAGA','03013','ALACANT/ALICANTE','ALICANTE','ESPAÑA',
    '12/01/1983','06/15/2008','pedsan70@yahoo.com',21,'H'
);

INSERT INTO USUARIOS VALUES(
    'diahue96','LSQZMC','DIANA','HUERTA VALIOS',
    'JOAQUIN SALAS','39011','SANTANDER','CANTABRIA','ESPAÑA',
    '04/25/1984','07/31/2009','diahue96@yahoo.com',395,'M'
);

INSERT INTO USUARIOS VALUES(
    'robrod74','<LQMLP','ROBERTO','RODRIGUEZ PARMO',
    'CASTILLO HIDALGO','51002','CEUTA','CEUTA','ESPAÑA',
    '06/28/1978','03/16/2009','robrod74@gmail.com',486,'H'
);

INSERT INTO USUARIOS VALUES(
    'milgar78','SF=UZ8','MILAGROSA','GARCIA ELVIRA',
    'PEDRALBA','28037','MADRID','MADRID','ESPAÑA',
    '04/12/1983','05/15/2008','milgar78@gmail.com',330,'M'
);

INSERT INTO USUARIOS VALUES(
    'frabar93','19JZ7@','FRANCISCA','BARRANCO RODRIGUEZ',
    'BALSAS, LAS','26006','LOGROÑO','RIOJA (LA)','ESPAÑA',
    '09/21/1986','02/16/2008','frabar93@gmail.com',75,'M'
);

INSERT INTO USUARIOS VALUES(
    'migarc93','AAFLTW','MIGUEL ANGEL','ARCOS ALONSO',
    'ISAAC ALBENIZ','04008','ALMERÍA','ALMERÍA','ESPAÑA',
    '03/01/1991','06/16/2010','migarc93@hotmail.com',23,'H'
);
```

TABLA JUEGOS

```
INSERT INTO JUEGOS VALUES(
    '1','Parchís',
    'El parchís es un juego de mesa derivado del pachisi y similar al ludo y al parcheesi'
);

INSERT INTO JUEGOS VALUES(
    '2','Oca',
    'El juego de la oca es un juego de mesa para dos o más jugadores'
);

INSERT INTO JUEGOS VALUES(
    '3','Ajedrez',
    'El ajedrez es un juego entre dos personas, cada una de las cuales dispone de 16 piezas móviles que se colocan sobre un to
);

INSERT INTO JUEGOS VALUES(
    '4','Damas',
    'Las damas es un juego de mesa para dos contrincantes'
);

INSERT INTO JUEGOS VALUES(
    '5','Poker',
    'El póquer es un juego de cartas de los llamados de "apuestas"'
);

INSERT INTO JUEGOS VALUES(
    '6','Chinchón',
    'El chinchón es un juego de naipes de 2 a 8 jugadores'
);

INSERT INTO JUEGOS VALUES(
    '7','Mus',
    'El mus es un juego de naipes, originario de Navarra, que en la actualidad se encuentra muy extendido por toda España'
);
```

```
INSERT INTO JUEGOS VALUES(
    '8','Canasta',
    'La canasta o rummy-canasta es un juego de naipes, variante del rummy'
);

INSERT INTO JUEGOS VALUES(
    '9','Dominó',
    'El dominó es un juego de mesa en el que se emplean unas fichas rectangulares'
);

INSERT INTO JUEGOS VALUES(
    '10','Pocha',
    'La pocha es un juego de cartas que se juega con la baraja española'
);

INSERT INTO JUEGOS VALUES(
    '11','Backgammon',
    'Cada jugador tiene quince fichas que va moviendo entre veinticuatro triángulos (puntos) según el resultado de sus dos dad
);

INSERT INTO JUEGOS VALUES(
    '12','Billar',
    'El billar es un deporte de precisión que se practica impulsando con un taco un número variable de bolas'
);
```

TABLA PARTIDA

```
INSERT INTO PARTIDAS VALUES(
    '1','Billar_migarc93_18/7','1','12','07/18/2011',
    TO_TIMESTAMP('00:47:40','HH24:MI:SS'),'migarc93'
);

INSERT INTO PARTIDAS VALUES(
    '2','Chinchón_mandia79_2/10','1','6','10/02/2011',
    TO_TIMESTAMP('01:47:40','HH24:MI:SS'),'mandia79'
);

INSERT INTO PARTIDAS VALUES(
    '3','Canasta_alibar52_26/2','0','8','02/26/2011',
    TO_TIMESTAMP('08:57:33','HH24:MI:SS'),'alibar52'
);

INSERT INTO PARTIDAS VALUES(
    '4','Damas_verbad64_16/3','1','4','03/16/2011',
    TO_TIMESTAMP('00:53:00','HH24:MI:SS'),'verbad64'
);

INSERT INTO PARTIDAS VALUES(
    '5','Chinchón_alibar52_9/9','1','6','09/09/2011',
    TO_TIMESTAMP('09:10:22','HH24:MI:SS'),'alibar52'
);

INSERT INTO PARTIDAS VALUES(
    '6','Oca_pedsan70_21/12','0','2','12/21/2011',
    TO_TIMESTAMP('18:53:17','HH24:MI:SS'),'pedsan70'
);

INSERT INTO PARTIDAS VALUES(
    '7','Canasta_encpay57_18/2','0','8','02/18/2011',
    TO_TIMESTAMP('09:41:02','HH24:MI:SS'),'encpay57'
);

INSERT INTO PARTIDAS VALUES(
    '8','Pocha_adofid63_26/10','1','10','10/26/2011',
    TO_TIMESTAMP('02:23:43','HH24:MI:SS'),'adofid63'
);

INSERT INTO PARTIDAS VALUES(
    '9','Damas_diahue96_25/6','1','4','06/25/2011',
    TO_TIMESTAMP('18:11:14','HH24:MI:SS'),'diahue96'
);

INSERT INTO PARTIDAS VALUES(
    '10','Parchís_encpay57_31/7','1','1','07/31/2011',
```

```
    TO_TIMESTAMP('21:21:36','HH24:MI:SS'),'encpay57'
);
```

TABLA UNEN

```
INSERT INTO UNEN VALUES('4','anamat56');
INSERT INTO UNEN VALUES('3','alecam89');
INSERT INTO UNEN VALUES('6','alecam89');
INSERT INTO UNEN VALUES('2','conmar76');
INSERT INTO UNEN VALUES('2','encpay57');
INSERT INTO UNEN VALUES('2','mandia79');
INSERT INTO UNEN VALUES('4','alibar52');
INSERT INTO UNEN VALUES('3','adofid63');
INSERT INTO UNEN VALUES('5','jesdie98');
INSERT INTO UNEN VALUES('8','pedsan70');
INSERT INTO UNEN VALUES('6','diahue96');
INSERT INTO UNEN VALUES('4','robrod74');
INSERT INTO UNEN VALUES('5','milgar78');
INSERT INTO UNEN VALUES('4','frabar93');
INSERT INTO UNEN VALUES('5','encpay57');
```



Autoevaluación

La tabla **UNEN** relaciona los usuarios con las partidas mediante **claves foráneas**.

¿Qué ocurrirá si se intenta insertar en UNEN un codigo_usuario que no existe en USUARIOS?

- ○ a. oracle insertará el registro sustituyendo automáticamente el usuario por null.
- ○ b. oracle rechazará la inserción por incumplirse la integridad referencial.
- ○ c. oracle creará automáticamente un nuevo usuario con ese login.

Incorrecta. Una **clave foránea** no sustituye automáticamente un valor inválido por NULL.

Correcta. La clave foránea exige que el **codigo_usuario** exista previamente en USUARIOS, por lo que la inserción será rechazada si no existe.

Incorrecta. Las claves foráneas no crean registros automáticamente en la tabla referenciada; su función es mantener la **integridad referencial**.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

Anexo II.- Creación y carga de tablas de la aplicación empresa en Oracle.

Este anexo explica cómo **crear un usuario de prácticas, ejecutar el script SQL y comprobar las tablas y datos cargados** en Oracle mediante línea de comandos o **SQL Developer**.

El primero paso será descargarte el script desde el siguiente enlace:

Script SQL: tablas y registros para realizar los ejercicios **(zip - 1,39 KB)**

Una vez descargado y descomprimido tendremos disponible el script que crea las tablas, restricciones y datos de ejemplo utilizados en los ejercicios.

Creación del usuario de prácticas

Crearemos un usuario denominado **ANA** para trabajar con las tablas de la aplicación EMPRESA.

En instalaciones actuales de Oracle es preferible crear **ANA como usuario local dentro de una PDB**. El prefijo **C##** se reserva, por defecto, para usuarios comunes del contenedor raíz, por lo que no es necesario utilizarlo en esta práctica.

El nombre del servicio dependerá de la instalación. Por ejemplo, Oracle Database XE utiliza habitualmente **XEPDB1**, mientras que Oracle Database Free utiliza **FREEPDB1** para su PDB predeterminada.

1.- Conexión como administrador

*Desde **SQL*Plus o SQLcl**, nos conectamos con un usuario administrador a la PDB en la que vamos a trabajar:*

```
<span class="eg">CONNECT</span><span> SYSTEM@localhost:</span><span class="ej">1521</span><span class="eg">/</<</span><span>serv
```

Introducimos la contraseña del usuario SYSTEM cuando se solicite.

2.- Creación del usuario ANA

*Creamos un **usuario local** dentro de la PDB:*

```
CREATE USER ana
IDENTIFIED BY "TuClaveDePractica1!"
DEFAULT TABLESPACE USERS
QUOTA 50M ON USERS;
```

La **cuota de tablespace** permite que ANA disponga de espacio para crear las tablas necesarias. Oracle permite asignar estas cuotas durante la creación del usuario.

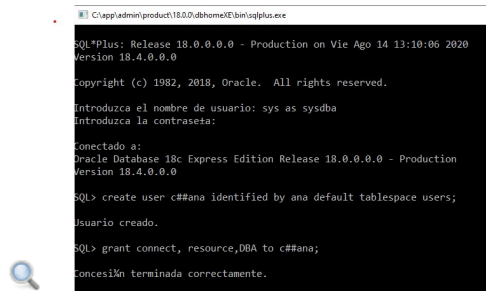
> La contraseña anterior es únicamente un **ejemplo para prácticas**. No debe reutilizarse como contraseña real.

3.- Concesión de permisos

Para ejecutar el script no es necesario otorgar el rol **DBA**. Basta con conceder los permisos necesarios para iniciar sesión y crear las tablas:

```
GRANT CREATE SESSION, CREATE TABLE TO ana;
```

De este modo, el usuario dispone únicamente de los **privilegios necesarios para realizar la práctica**.



Elaboración propia. *Creación del usuario c##ana desde SQLPlus.* (CC BY-NC-SA)

Ejecución del script BD04_CONT_R07_02_CORREGIDO.sql

El script creará las tablas y cargará los datos necesarios para realizar los ejercicios.

Podemos ejecutarlo de dos formas:

1. **Desde línea de comandos**, utilizando SQL*Plus o SQLcl.
2. **Desde un entorno gráfico**, utilizando SQL Developer.

Desde SQL*Plus o SQLcl

*Nos conectamos con el usuario **ANA** a la misma PDB:*

```
CONNECT ana@localhost:1521/<servicio_PDB>
```

Introducimos la contraseña cuando se solicite.

*Para ejecutar el script utilizamos el carácter **@** seguido de la ruta del archivo:*

```
@"C:\ruta\BD04_CONT_R07_02_CORREGIDO.sql"
```

Tanto SQL*Plus como SQLcl permiten ejecutar scripts mediante este mecanismo; SQLcl también admite el comando **@** para ejecutar archivos SQL.

Una vez ejecutado, aparecerán mensajes indicando la **creación de las tablas** y la **inserción de los registros**.

Para guardar definitivamente las inserciones ejecutamos:

```
COMMIT;
```

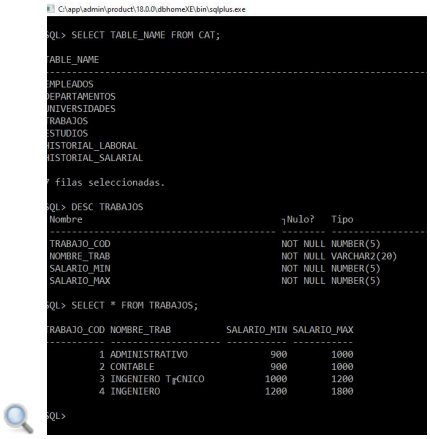
Comprobación de las tablas

Para comprobar que el script se ha ejecutado correctamente podemos consultar las tablas existentes:

```
SELECT TABLE_NAME
FROM CAT;
```

Deberán aparecer las tablas:

- **EMPLEADOS**.
- **DEPARTAMENTOS**.
- **UNIVERSIDADES**.
- **TRABAJOS**.
- **ESTUDIOS**.
- **HISTORIAL_LABORAL**.
- **HISTORIAL_SALARIAL**.



Elaboración propia. *Captura de pantalla con ejecución sentencias SELECT y comando DESC.* (CC BY-NC-SA)

Ejecución con SQL Developer

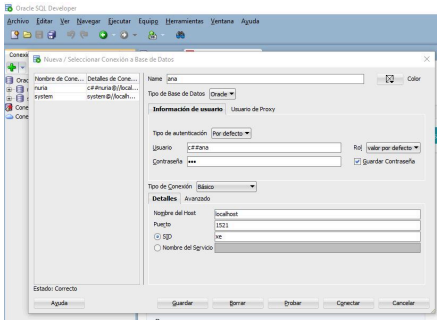
También podemos realizar el proceso desde **Oracle SQL Developer**.

*Creamos una conexión para el usuario **ANA** utilizando:*

- **Nombre de conexión.** ana.
- **Usuario.** ana.
- **Contraseña.** La utilizada al crear el usuario.
- **Host.** localhost.
- **Puerto.** 1521.
- **Tipo de conexión.** Básico.
- **Nombre del servicio.** El correspondiente a la PDB, por ejemplo XEPDB1 o FREEPDB1.

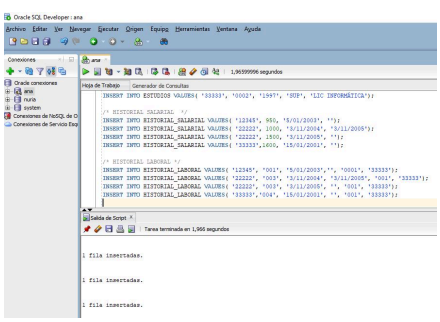
Para un usuario local es importante conectarse al **servicio de la PDB**, no al contenedor raíz.

Una vez creada la conexión, abrimos el archivo **BD04_CONT_R07_02_CORREGIDO.sql** en la hoja de trabajo y ejecutamos el script completo mediante **F5**.



Elaboración propia. *Creación conexión en SQLDeveloper para c##ana* (CC BY-NC-SA)

Abre la conexión y ejecuta el script desde el entorno gráfico **SqlDeveloper**, copiando y pegando el contenido del fichero de texto en el editor como si fuera una sola orden y ejecútalo con F5.

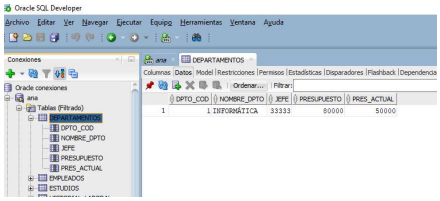


Elaboración propia. *Ejecución script en SQLDeveloper.* (CC BY-NC-SA)

También podemos ejecutar:

```
SELECT TABLE_NAME
FROM CAT;
```

Si seleccionamos una tabla desde el árbol de conexiones podremos consultar sus **columnas, restricciones y datos**.



Elaboración propia. *Datos de una tabla* (CC BY-NC-SA)

Para visualizar directamente su contenido también podemos utilizar una consulta SELECT:

```
SELECT *
FROM DEPARTAMENTOS;
```

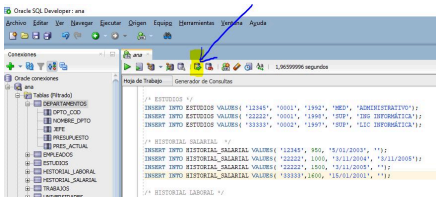
Confirmación de las inserciones

Después de ejecutar el script debemos **confirmar las inserciones** para que los cambios queden guardados.

Podemos hacerlo ejecutando:

```
COMMIT;
```

o utilizando el botón correspondiente de **SQL Developer**.



Elaboración propia. *Confirmación (COMMIT) de la operación* (CC BY-NC-SA)

Una vez realizado este proceso, el usuario **ANA** dispondrá de todas las tablas y datos necesarios para practicar las consultas SQL de los ejercicios.

Autoevaluación

Un alumno va a crear el usuario **ANA** exclusivamente para cargar las tablas y realizar las consultas de la práctica.

¿Qué configuración resulta más adecuada?

- ○ a. crear c##ana en el contenedor raíz y concederle el rol dba.
- ○ b. crear ana como usuario local en la pdb y conceder únicamente los privilegios necesarios.
- ○ c. utilizar siempre el usuario sys para crear tablas y realizar todos los ejercicios.

Incorrecta. El prefijo **C##** corresponde normalmente a usuarios comunes y el rol **DBA** concede muchos más privilegios de los necesarios para esta práctica.

Correcta. Un **usuario local en la PDB** permite trabajar con las tablas de la práctica sin otorgar permisos administrativos innecesarios.

Incorrecta. SYS es un usuario administrativo y no debe utilizarse como usuario habitual para crear y consultar las tablas de los ejercicios.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

Anexo III. Ejercicios SQL propuestos con posible solución

A programar se aprende programando, por ello te proponemos dos bloques de ejercicios para que practiques los contenidos trabajados.

BLOQUE I. EJERCICIOS SQL

A continuación, y sobre la **base de datos de Juegos Online**, resuelve y prueba en tu ordenador las siguientes sentencias SQL:

1. Nombre y apellidos de los usuarios con crédito entre **200 y 400**, ordenados por crédito descendente.
2. ¿Cuántos usuarios son mujeres?
3. Nombre y apellidos de los usuarios que tienen correo en **Hotmail**, ordenados por apellido y nombre.
4. Suma del crédito de los usuarios de la provincia de **Barcelona**.
5. Nombre, apellidos y fecha de nacimiento del usuario de **más edad**.
6. Listado con la suma del crédito de los usuarios de cada provincia, ordenado por provincia.
7. Provincias en las que la suma del crédito de los usuarios es **menor de 200**.
8. ¿Cuál es la provincia en la que la suma de crédito de los usuarios es la **mayor de todas**?
9. Nombre y apellidos del usuario que ha creado cada una de las partidas, indicando **de qué juego es**.
10. Juegos de los que **no hay ninguna partida registrada**.
11. Listado de juegos de los que hay alguna **partida en funcionamiento**.
12. Listado con el nombre y apellidos de cada usuario y el **número de partidas en las que está jugando**, ordenado de mayor a menor.

*En el siguiente enlace tienes una **posible solución**:*

Solucion Ejercicios SQL Anexo III (pdf - 47,85 KB)

BLOQUE II. EJERCICIOS SQL

Para este segundo bloque utilizaremos una **base de datos de práctica adicional**, distinta de la aplicación EMPRESA utilizada anteriormente en la unidad.

Esta base de datos contiene información sobre **centros, departamentos y empleados**, e incluye las sentencias necesarias para crear y cargar las tablas, además de una relación de **20 ejercicios SQL**.

Descarga el siguiente archivo, ejecuta las sentencias de creación y carga en Oracle y realiza posteriormente los ejercicios propuestos:

Creación y carga Base de datos empresa gestión empleados

En este enlace tienes una posible solución a los requerimientos SQL anteriormente planteados.

Posible Solución Consultas

> **Nota:** esta base de datos se utiliza únicamente para este bloque de ejercicios y **no sustituye a la aplicación EMPRESA empleada anteriormente en la unidad**.

El archivo corregido mantiene las tablas **CENTROS, DEPARTAMENTOS y EMPLEADOS** y los 20 ejercicios originales, pero ajusta el orden de creación y carga para que las claves foráneas puedan aplicarse correctamente.

Ejercicios propuestos

1. Hallar la comisión, el nombre y el salario de los empleados con **más de tres hijos**, ordenados por comisión y, dentro de comisión, alfabéticamente.
2. Obtener los nombres de los departamentos que **no dependen de otros**.
3. Obtener, por orden alfabético, los nombres y salarios de los empleados cuyo salario esté comprendido **entre 1250 y 1300 euros, ambos incluidos**.
4. Mostrar los datos de los empleados que cumplen la condición anterior o tienen **al menos un hijo**.
5. Mostrar para cada empleado el **número de meses que lleva en la empresa** junto con su nombre.
6. Calcular qué empleados llevan **más de 35 años en la empresa** y mostrar todos sus datos.
7. Hallar, por orden alfabético, los nombres de los empleados tales que, si se les da una **gratificación de 10 euros por hijo**, el total de esta gratificación no supera la centésima parte del salario.
8. Hallar, por orden de número de empleado, el nombre y el **salario total (salario más comisión)** de los empleados cuyo salario total supera los 1300 euros mensuales.
9. Obtener, por orden alfabético, los nombres de los departamentos que no contengan las palabras **Dirección** ni **Sector**.
10. Obtener, por orden alfabético, los nombres de los departamentos que, o bien tienen **directores en funciones y su presupuesto no excede los 5 mil euros**, o bien no dependen de ningún otro departamento.
11. Hallar el **número de empleados** de toda la empresa.
12. Hallar cuántos departamentos existen y el **presupuesto anual medio** para el global de todos los departamentos.
13. Hallar el número de empleados y de **extensiones telefónicas distintas** del departamento 112.
14. Obtener los mismos datos para los departamentos que **no tienen director en propiedad**.
15. Mostrar los datos de los empleados que trabajan en un centro cuya dirección contiene **Atocha** y tienen exactamente **dos hijos**. No utilizar subconsultas anidadas.
16. Extraer los nombres de las **parejas de empleados, sin repetir**, que trabajan en el mismo centro y tienen el mismo salario base. Mostrar también el salario y no utilizar subconsultas anidadas.
17. Utilizar operaciones de conjuntos para extraer los códigos de los departamentos que **no hacen de departamento jefe**.

18. Utilizar operaciones de conjuntos para extraer los códigos de los departamentos que **sí hacen de departamento jefe** de algún otro departamento.
19. Extraer un listado donde aparezcan el **código y nombre de los departamentos**, junto con el **código y nombre de los centros** en los que están situados.
20. Utilizar operaciones de conjuntos para obtener los nombres y salarios de los empleados cuyo salario esté comprendido **entre 1250 y 1300 euros, ambos incluidos**, y que tengan al menos un hijo.

Licencia: licencia propietaria